# DIGITAL TWIN ASSISTED DYNAMIC NETWORK SLICING USING SSL, TGNN AND MARL FOR AI-NATIVE 6G NETWORKS

Project Record: Work Completed up to October 2026

Submitted in partial fulfilment for the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)

by

**KEERTHIVASAN S V (CB.SC.U4AIE23037)**

**KRISH S**

**SRIRANJANA C**

**THRISHALA S N**

Under the guidance of Dr. Sundaresan




DEPARTMENT OF ARTIFICIAL INTELLIGENCE

**AMRITA VISHWA VIDYAPEETHAM**

COIMBATORE CAMPUS (INDIA)

October 2026

# Abstract

Network slicing lets a 6G network carry very different services, namely enhanced mobile broadband (eMBB), ultra-reliable low-latency communication (URLLC) and massive machine-type communication (mMTC), on shared infrastructure. Most slicing schemes in the literature react to demand after it has changed. This project studies whether predicting traffic before allocating bandwidth helps, using a pipeline of self-supervised representation learning (SSL), a temporal graph neural network (TGNN) for forecasting and multi-agent reinforcement learning (MARL) for allocation, tested inside a Digital Twin of the network.

This record documents the work completed between the July 2026 kickoff and 22 October 2026. We reviewed the literature across the four pillars, collected ten public datasets and implemented every module. Following the review of 12 October, we found that no public dataset joins topology, per-slice traffic, quality of service and allocation actions, so we built a unified slicing environment on real Telecom Italia Milan traffic with latency calibrated on a public 6G RAN telemetry dataset, and a Digital Twin dataset of 393.6 million rows and 122 columns. On a held-out week, a TGNN trained on 39 days beats repeating the last value by 18%, 33% and 31% for eMBB, URLLC and mMTC across all 10,000 Milan cells, and improves an expert allocation rule by about 0.03 reward on every test day. A single centralised agent scores 0.44, 0.34 and 0.28 at 9, 25 and 100 cells, while a parameter-shared MARL controller scores 0.93, 0.83 and 0.86 with a constant 19,332-parameter actor; trained for 4,000 episodes with checkpoints chosen on validation days, MARL reaches 0.90 at 25 cells, 0.026 above an expert rule whose thresholds were tuned on the training data, on all seven test days.

Several results are negative and are reported as such: self-supervised pretraining gave no reliable gain over raw features on 5G-NIDD at any label budget, MARL only ties the tuned expert rule with a short training budget, trained controllers lose to the tuned rule when the traffic mix changes without retraining, and the effect of the forecast on MARL is not statistically established. The record closes with the limitations of the current evidence and a publication plan for the guide's approval.

# Contents

Abstract ..... i
Contents ..... ii
Abbreviations ..... iv
Chapter 1 Introduction ..... 1
1.1 Background ..... 1
1.2 Problem statement ..... 1
1.3 Proposed approach ..... 1
1.4 Objectives ..... 2
1.5 Organisation of this record ..... 2
Chapter 2 Timeline and Team ..... 3
2.1 Timeline of work ..... 3
2.2 Team and responsibilities ..... 3
2.3 Alignment with the School assessment policy ..... 4
Chapter 3 Literature Survey ..... 5
3.1 Scope and coverage ..... 5
3.2 Datasets used in the reviewed papers ..... 5
3.3 Research gap ..... 6
Chapter 4 Datasets ..... 7
4.1 Datasets collected ..... 7
4.2 Is there a unified public dataset? ..... 7
Chapter 5 Module Implementation up to the First Review ..... 8
5.1 Results of each module ..... 8
5.2 Errors found and corrected ..... 10
Chapter 6 Review of 12 October and the Unified Environment ..... 12
6.1 Comments from the review ..... 12
6.2 Design of the unified environment ..... 12
6.3 Policies compared ..... 13
Chapter 7 Experiments and Results ..... 14
7.1 Experimental set-up ..... 14
7.2 Forecasting the whole city ..... 14
7.3 Value of the forecast to the expert rules ..... 15
7.4 MARL compared with a single centralised agent ..... 16
7.5 Training MARL long enough ..... 17
7.6 Does the forecast help MARL? ..... 19
7.7 A change in the traffic mix ..... 19
7.8 Self-supervised learning revisited ..... 20

7.9 Controllers across the whole city ..... 21
Chapter 8 The Digital Twin Dataset ..... 23
8.1 Overview ..... 23
8.2 Scenarios ..... 23
8.3 Energy saving across the city ..... 24
8.4 Checks on the generated data ..... 24
Chapter 9 Prototype: the Slice Twin Monitor ..... 25
9.1 Purpose ..... 25
9.2 Views ..... 25
Chapter 10 Limitations of the Current Evidence ..... 27
10.1 Statistical strength ..... 27
10.2 Scope of the MARL result ..... 27
10.3 Model fidelity ..... 27
10.4 Negative results ..... 27
10.5 The Digital Twin claim ..... 28
Chapter 11 Publication Plan and Pending Work ..... 29
11.1 Review paper ..... 29
11.2 Original paper ..... 29
11.3 Questions for the guide ..... 30
11.4 Pending work ..... 30
References ..... 32
Appendix A: Literature Review Notes ..... 33
Appendix B: Columns of the Digital Twin Dataset ..... 62

*(If page numbers do not appear, right-click the table and choose Update Field.)*

# Abbreviations

| | |
|---|---|
| **5G-NIDD** | 5G Network Intrusion Detection Dataset |
| **APC** | Article processing charge |
| **CQI** | Channel quality indicator |
| **DDPG** | Deep deterministic policy gradient |
| **DT** | Digital Twin |
| **eMBB** | Enhanced mobile broadband |
| **GNN** | Graph neural network |
| **GRU** | Gated recurrent unit |
| **MADDPG** | Multi-agent deep deterministic policy gradient |
| **MARL** | Multi-agent reinforcement learning |
| **mMTC** | Massive machine-type communication |
| **PRB** | Physical resource block |
| **QoS** | Quality of service |
| **RAN** | Radio access network |
| **SINR** | Signal-to-interference-plus-noise ratio |
| **SLA** | Service level agreement |
| **SSL** | Self-supervised learning |
| **TGNN** | Temporal graph neural network |
| **URLLC** | Ultra-reliable low-latency communication |

# Chapter 1
# Introduction

## 1.1 Background

Sixth-generation networks are expected to host services with conflicting requirements on the same physical network. Video streaming and extended reality need high throughput, industrial control and vehicle coordination need latencies of a few milliseconds with very high reliability, and sensor networks need to connect very large numbers of low-rate devices. Network slicing addresses this by giving each service class its own logical share of the radio and core resources, with its own service level agreement (SLA).

Deciding how large each slice should be is difficult because demand changes continuously and differs from cell to cell. Static splits waste capacity when a slice is idle and break SLAs when it surges. Learning-based controllers adapt better, but in most published work they respond to the state the network is already in, so any reallocation lags behind the change in demand.

## 1.2 Problem statement

The project asks whether a slicing controller that is told what demand will look like in the next interval allocates bandwidth better than one that only sees current demand, and whether a decentralised multi-agent controller is a better fit for this task than a single centralised one. A secondary aim is to test these controllers safely in a Digital Twin before they would be applied to a real network.

## 1.3 Proposed approach

The proposed pipeline has four stages. An SSL encoder learns compact representations of network telemetry from unlabelled data. A TGNN, which combines GraphSAGE aggregation over the cell graph with a recurrent update over time, forecasts the next interval's demand for each slice. A MARL controller, with one agent per cell, splits each cell's capacity between eMBB, URLLC, mMTC and an idle share. The Digital Twin is a virtual copy of the network in which these allocations are trained, evaluated and, ideally, approved before deployment.

**Table 1.1:** Stages of the proposed pipeline

| Stage | Role | Output |
|---|---|---|
| Self-supervised learning | Learns representations of telemetry by masking features and reconstructing them | 64-dimensional embedding per node and time step |
| Temporal GNN | Forecasts next-interval demand per cell and slice from the cell graph and recent history | Predicted demand |

| Stage | Role | Output |
|---|---|---|
| Multi-agent RL | Each cell agent decides how to split its bandwidth between slices | Slice shares per cell |
| Digital Twin | Virtual network used to train, test and check allocations; supports what-if scenarios | Evaluation, scenario data |

## 1.4 Objectives

1. Survey the literature on Digital Twins, SSL, graph-based forecasting and MARL for network slicing, and identify the gap.
2. Collect and prepare public datasets suitable for each stage of the pipeline.
3. Implement and evaluate each module on real data against simple baselines.
4. Build an environment in which prediction and allocation can be tested together on real traffic.
5. Deliver a working prototype and prepare research papers from the results.

## 1.5 Organisation of this record

Chapter 2 gives the timeline and team roles. Chapter 3 summarises the literature survey and Chapter 4 the datasets. Chapter 5 covers the module implementations up to the first review. Chapter 6 describes the response to the review of 12 October and the unified environment, and Chapter 7 the experiments run in it. Chapter 8 describes the Digital Twin dataset and Chapter 9 the prototype. Chapter 10 discusses limitations, and Chapter 11 the publication plan and pending work. The full literature review notes are in Appendix A.

# Chapter 2
# Timeline and Team

## 2.1 Timeline of work

Table 2.1 lists the main milestones. Dates are taken from the meeting minutes and the history of the project repository (github.com/FYP-A09/6G).

**Table 2.1:** Project milestones in 2026

| Date | Milestone | Outcome |
|---|---|---|
| 11 July | Kickoff meeting with the guide | Problem, novelty position and publication roadmap agreed; each member to review five to six papers |
| July to September | Literature survey | Papers reviewed across the four pillars; candidate datasets studied |
| 16 September | Repository and datasets | Repository created; eight datasets downloaded; download scripts written for the two large datasets |
| 17 September | First review deliverables | Requirements, design and task documents per member; interface contracts and architecture diagram; prototypes of each module; end-to-end orchestrator |
| 17 and 18 September | Module implementations merged | SSL pipeline, TGNN graph construction and training, MARL slicing environment; nine bugs fixed in a code review |
| 18 September | Simulator and training | Simu5G, OMNeT++ and INET built under WSL2; reference NR run completed; first MADDPG training |
| 30 September | Forecasting on real traffic | TGNN trained on real Milan traffic; data loader and evaluation split corrected |
| 7 October | Correction of the MARL environment | Slice attribution bug fixed and all affected results re-run |
| 12 October | Review with the guide | TGNN shown to beat persistence over a full day; SSL ablation presented; four review comments received |
| 12 to 16 October | Response to the review | Dataset audit, unified environment, MARL versus single-agent study, forecast value, dataset generator |
| 15 and 16 October | Large-scale data generation | 393.6 million-row Digital Twin dataset; city-wide forecaster; 25 automated tests passing |
| 19 and 20 October | Prototype and planning | Monitoring prototype with block and city views; new data samples; literature search for publications |
| 21 and 22 October | Retraining and re-testing | Forecaster retrained on 39 days; SSL re-evaluated on de-duplicated data with two pretraining methods; MARL retrained with 40 days of data, longer budgets, tuned baselines and a held-out week; all documents updated |

## 2.2 Team and responsibilities

The work was divided by pillar so that each member's literature review, datasets and implementation formed one coherent part of the system (Table 2.2). Shared items, such as the novelty statement and the work plan, were prepared together.

**Table 2.2:** Division of work

| Member | Pillar | Main contributions |
|---|---|---|
| Keerthivasan S V | Digital Twin and integration | Repository and data pipeline, interface contracts, orchestrator, simulator set-up, unified environment, MARL versus single-agent study, city-wide forecaster, Digital Twin dataset, monitoring prototype |
| Krish S | MARL and slicing | Problem formulation as a decentralised POMDP, B5G slicing environment, MADDPG training and tuning, reward design |
| Sriranjana C | Self-supervised learning | 5G-NIDD preprocessing, masked-reconstruction encoder, embeddings for NIDD, Milan and NeversNet5G, embedding evaluation |
| Thrishala S N | Temporal GNN | Graph construction for NeversNet5G, Milan and B5G, the TGNN model, forecasting on Milan traffic |

## 2.3 Alignment with the School assessment policy

The School of AI policy for final year projects (AY 2026-27) requires B.Tech projects to be product-oriented and allots 10 marks for a working product or prototype and 5 marks for a paper, industry collaboration, patent or startup. The working prototype for this project is the Slice Twin monitor described in Chapter 9, supported by the code, environment and dataset in the repository. The paper component is addressed by the plan in Chapter 11. The policy also returns final reports with more than 15% plagiarism or more than 20% AI-generated content, which the team will check before submitting the thesis.

# Chapter 3
# Literature Survey

## 3.1 Scope and coverage

Each member reviewed papers from their own pillar, published mainly between 2022 and 2026 in IEEE journals and conferences, with some arXiv and TechRxiv preprints. For every paper we recorded the problem, the method, the dataset or simulation tool, the results, the research gap and the relevance to this project. The complete notes are reproduced in Appendix A, and Table 3.1 summarises the coverage.

**Table 3.1:** Coverage of the literature survey

| Pillar | Representative papers | Recurring gap |
|---|---|---|
| Digital Twin | Native network Digital Twin architecture for 6G (2025); AI-driven Digital Twins for slicing with non-terrestrial networks (2025); slicing-aware Digital Twins with graph attention (2025); GNN-based Digital Twin for slicing (2022); self-optimising 6G networks through Digital Twin intelligence on Milan data (2026) | Twins mostly react to the current state; validation on private or self-built data |
| Self-supervised learning | Survey of SSL for wireless networks (2024); Net2Net position prediction (2025); SSL for encrypted traffic classification in SDN (2022) | Rarely applied to slicing telemetry; evaluated on non-telecom or self-generated data |
| Graph-based forecasting | GraphSAGE-based Digital Twin for latency prediction (2022); self-supervised spatio-temporal GNN for traffic (2023) | Static graphs; road-traffic rather than cellular benchmarks |
| RL and MARL for slicing | Multi-agent RAN slicing for 6G (2026); GAN-enhanced DRL (2026); MARL for vehicular slicing (2023); DRL for end-to-end slicing (2023); slice mobility (2022); slicing for AI services (2025) | Reactive decisions; small cell clusters; own simulators |

## 3.2 Datasets used in the reviewed papers

At the review of 12 October the guide asked which datasets the papers use. Table 3.2 answers this for the sixteen papers in the repository's survey notes.

**Table 3.2:** Data used by the reviewed papers

| No. | Paper | Data used | Kind |
|---|---|---|---|
| 1 | Ali and Arslan (2025), AI-driven Digital Twins for slicing | Python 5G simulator, 50 UEs, 20 MHz | Own simulator |
| 2 | Li et al. (2026), multi-agent RAN slicing | NS-3 RAN slicing simulator | Own simulator |
| 3 | Chowdhury et al. (2025), semantic slicing | None (conceptual) | None |
| 4 | John (2026), GAN-enhanced DRL | Custom 6G simulator with GAN traffic | Own simulator |
| 5 | MARL for vehicular slicing (2023) | Mininet with Ryu SDN controller | Own emulator |
| 6 | Native network Digital Twin (2025) | Operator-inspired scenario | Private |
| 7 | Slicing-aware Digital Twins (2025) | Docker UPF and extended OMNeT++ | Own testbed |
| 8 | GNN-based Digital Twin (2022) | NSFNET, GEANT2, synthetic topologies | Public topologies |

| No. | Paper | Data used | Kind |
|---|---|---|---|
| 9 | SSL for wireless networks (2024) | None (survey) | None |
| 10 | Net2Net (2025) | Cora, Citeseer, DBLP, PubMed and others | Non-telecom graphs |
| 11 | SSL for encrypted traffic (2022) | Mininet and D-ITG traffic | Own generated |
| 12 | Slice mobility for 6G (2022) | Discrete-time simulation | Own simulator |
| 13 | Slicing for AI services (2025) | Mobile-health dataset | Non-telecom |
| 14 | Self-supervised spatio-temporal GNN (2023) | METR-LA, PEMS-BAY, PEMS03/04/07/08 | Road traffic |
| 15 | DRL for end-to-end slicing (2023) | OpenAirInterface and FlexRAN testbed | Own testbed |
| 16 | Self-optimising 6G via Digital Twin (2026) | Telecom Italia Milan | Public cellular |

Eight of the sixteen papers generate their own data with a simulator, emulator or testbed, and only one uses public cellular traffic (the Milan dataset, which we also use). None uses a public dataset that combines topology, per-slice traffic, QoS and allocation decisions. Generating data is therefore the usual practice in this area, which supports the approach taken in Chapter 6.

## 3.3 Research gap

Across the reviewed work, the components of our pipeline appear separately: Digital Twins combined with reinforcement learning, MARL for slicing, and GNN-based traffic prediction. We did not find a paper that combines SSL, temporal graph forecasting, a Digital Twin and MARL for slicing, and most controllers are reactive. The guide advised at the kickoff that novelty claims must be backed by evidence from the literature; Chapter 11 revisits this claim in the light of a wider search.

# Chapter 4
# Datasets

## 4.1 Datasets collected

**Table 4.1:** Datasets used in the project

| Dataset | Size | Contents | Use in this project |
|---|---|---|---|
| Telecom Italia Milan | 62 days, 10,000 cells, 10-minute intervals | SMS, call and internet activity on a 100 x 100 grid of 235 m cells | Real traffic for SSL, TGNN, the unified environment and the Digital Twin dataset |
| 5G-NIDD | 1,215,890 flows | Labelled benign and attack flows from a 5G network | Training and evaluation of the SSL encoder |
| NeversNet5G | 716 UE files, about 27 GB, 19 gNodeBs | Vehicular 5G NR telemetry produced with Simu5G and SUMO | UE to gNodeB topology (695 nodes) and telemetry replay |
| B5G network slicing | 7,984 samples | Transport topologies with eMBB, URLLC and mMTC flows | First MARL environment |
| 6G RAN telemetry | 30,000 windows | PRB load, latency percentiles, SLA flag and reward | Calibration of latency against load |
| DeepSlice, Secure5G, Kaggle slicing sets, Botez 6G slicing | Small | Slice labels and QoS budgets | Slice-type benchmarks |

## 4.2 Is there a unified public dataset?

A dataset that connects every stage of the pipeline would need, in one record, the network topology, per-cell traffic over time, per-slice demand, QoS measurements, the SLA outcome and the allocation action with its reward. We checked each dataset we hold and searched the web in October 2026. None contains all of these. The closest are the B5G slicing dataset, which covers only the transport network and has no record of allocation actions, and published O-RAN slicing traces, which come from single-cell testbeds and whose release we could not confirm. Following the guide's advice, we therefore built a unified dataset synthetically, grounded in real data wherever real data exists (Chapters 6 and 8).

# Chapter 5
# Module Implementation up to the First Review

## 5.1 Results of each module

Table 5.1 summarises what was built for each module and how it performed on held-out real data. Each result is compared with a simple baseline, because a model that cannot beat a baseline has not yet shown value.

**Table 5.1:** Module results on real data

| Module | Implementation | Result |
|---|---|---|
| SSL on 5G-NIDD | Masked-reconstruction encoder producing 64-dimensional embeddings, trained on all 1.2 million flows | First evaluation: macro-F1 0.708 with a linear classifier, equal to raw features. Superseded: the evaluation had duplicate flows and leaked identifiers; see Section 7.8 |
| SSL on Milan | Masked-reconstruction encoder on the full Milan release | Trains; embeddings used as TGNN input |
| TGNN | GraphSAGE with GRU, next-step forecast on 400 real Milan cells over a full day | First version: validation MSE 0.000616 against 0.000720 for repeating the last value (about 14% better, 28 windows). Superseded by the city-wide forecaster trained on 39 days; see Section 7.2 |
| MARL on B5G | B5G environment, MADDPG, train, validation and test split with tuning | Held-out reward 0.447 against 0.372 for a heuristic and 0.312 for equal split; the reward is close to trivial because B5G has no latency or loss data |
| Digital Twin loop | B5G sample, TGNN, MARL action and an approve or reject gate | Runs end to end; approves no actions because B5G lacks the telemetry needed for the SLA check |
| Simulator | OMNeT++ 6.4.0, INET 4.7.0 and Simu5G 1.7.0 under WSL2 | Reference NR voice run: 28,092 events, 93 of 93 packets delivered, MOS 4.41; the project's own 19-gNodeB scenario is not yet connected |
| Topology check | Nearest-gNodeB association on NeversNet5G, 118 UEs | Correlation between distance and SINR of -0.355; every UE within its cell's range |

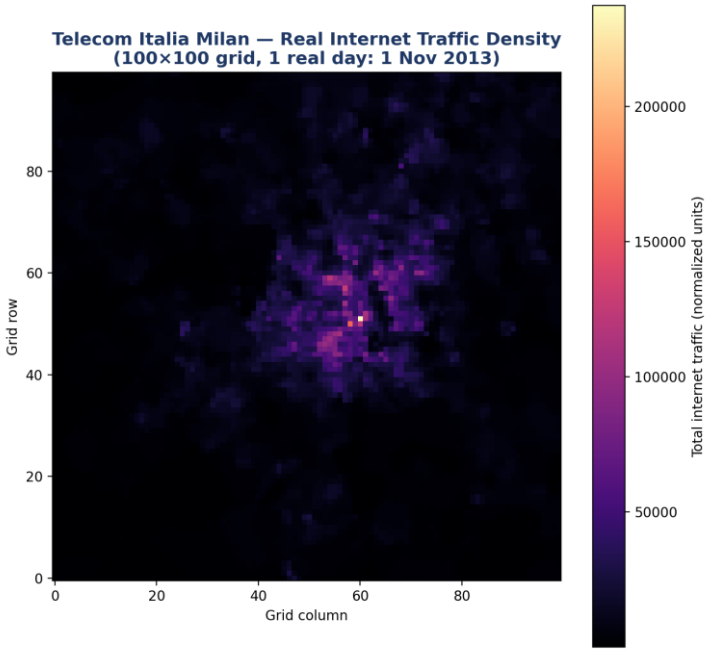



**Figure 5.1:** Real traffic on the Milan grid

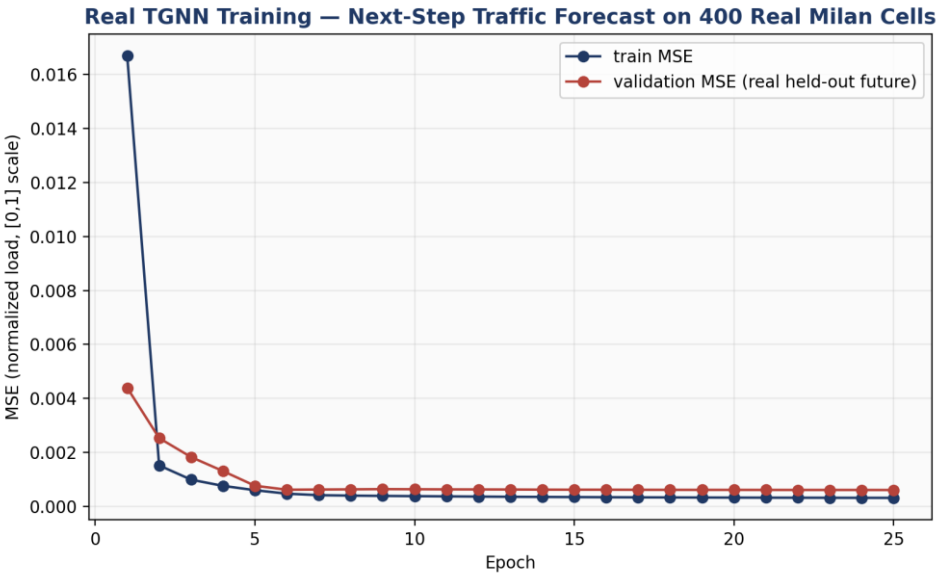



**Figure 5.2:** TGNN training and validation error on real Milan traffic

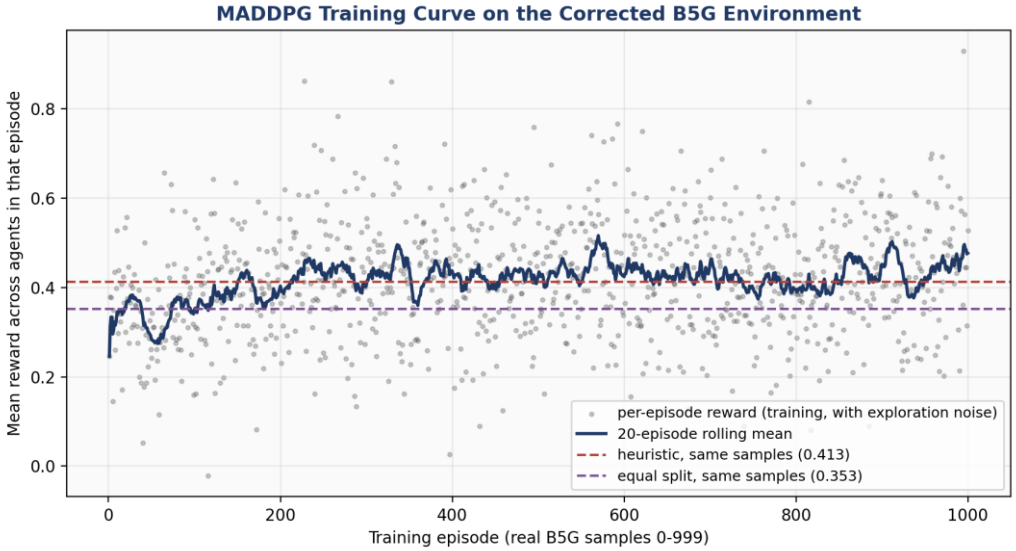



**Figure 5.3:** MADDPG training reward on the B5G slicing environment

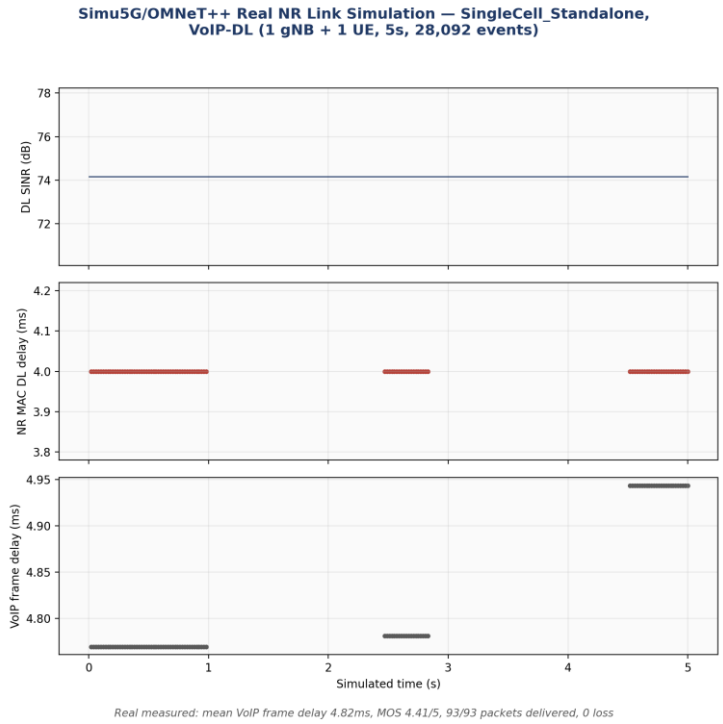



**Figure 5.4:** Reference Simu5G NR run

## 5.2 Errors found and corrected

Two errors invalidated early results and were corrected before the first review. The 5G-NIDD file is stored in blocks of identical labels, so a sequential train-test split placed 99.88% of one class in the test set and produced an accuracy near 1%; we replaced it with a stratified split. In the B5G environment, agents were keyed by their position in the slice list rather than by slice, so most rewards were computed against the wrong slice. We fixed the indexing, added a regression test and re-ran all MARL experiments. A code review of the merged module branches on 18 September also found and

fixed nine smaller correctness and efficiency problems, including a mismatch between mMTC and mIoT labels.

# Chapter 6
# Review of 12 October and the Unified Environment

## 6.1 Comments from the review

At the review of 12 October the guide made four comments: look for unified data that connects every stage; justify the use of MARL instead of a single agent; if unified data is not available, create it synthetically; and report which datasets the reviewed papers use. Table 6.1 lists how each was addressed.

**Table 6.1:** Response to the review comments

| Comment | Response | Where |
|---|---|---|
| Datasets used in papers | Audited the sixteen papers in the survey notes | Section 3.2 |
| Unified data | Checked every dataset held and searched the web; none is unified | Section 4.2 |
| Create it synthetically | Built the unified environment below and a large dataset from it | Section 6.2 and Chapter 8 |
| Why MARL | Controlled comparison of MARL and a single centralised agent, and with a tuned rule | Sections 7.4 and 7.5 |

## 6.2 Design of the unified environment

The environment joins, for each cell and each 10-minute interval, real traffic on a real spatial layout, per-slice demand, an allocation action and the resulting QoS, SLA outcome and reward. It is implemented in the src/unified directory of the repository. Table 6.2 states which parts are real, which are calibrated and which are modelling assumptions.

**Table 6.2:** Components of the unified environment

| Component | Status | Source or rule |
|---|---|---|
| Traffic per cell and interval | Real | Telecom Italia Milan |
| Topology | Real | Milan grid layout with 4-connected neighbours |
| Slice demand | Modelled | Internet activity mapped to eMBB, calls to URLLC and SMS to mMTC, scaled so that each slice's 95th percentile is 80%, 10% and 5% of a 100 Mbps cell |
| Latency and jitter against load | Calibrated | Curves fitted to the 6G RAN telemetry dataset (30,000 windows) |
| Inter-cell interference | Modelled | Usable capacity = 100 Mbps / (1 + 0.3 x mean neighbour load) |
| Loss and SLA | Rules | Targets for eMBB 50 ms latency, 20 ms jitter, 0.99 reliability; URLLC 5 ms, 1 ms, 0.99999; mMTC 100 ms, 50 ms, 0.99 |
| Action | Agent output | Shares of each cell's capacity for eMBB, URLLC, mMTC and idle, applied to the next interval |
| Reward | Design choice | 0.7 x weighted SLA score + 0.3 x fraction of demand served - 0.1 x reconfiguration churn |

To check realism, we compared the eMBB SLA rate against load with the SLA label in the reference telemetry. A deterministic latency model matched poorly, with a mean gap of 0.086 across fifteen load bins. Adding lognormal variation in latency with a single parameter (sigma = 0.21) fitted to that curve reduced the gap to 0.021. Since the parameter was fitted on the same curve, this is a goodness-of-fit measure rather than an independent validation.

## 6.3 Policies compared

Besides the learned agents, we evaluated an equal split, an allocation proportional to demand, and an expert policy that gives URLLC and mMTC enough capacity to stay below their SLA-safe utilisation and assigns the remainder to eMBB. The expert policy was run with three inputs: current demand, the TGNN forecast, and the true next-interval demand, which serves as an upper bound.

# Chapter 7
# Experiments and Results

## 7.1 Experimental set-up

The experiments use real Milan traffic. Learned models are trained on days 1 to 40 of the recorded period (1 November to 10 December 2013), a validation block of days 41 to 44 is used to choose checkpoints and tune thresholds, and every result in this chapter is measured on a held-out week, 16 to 22 December 2013 (days 45 to 51), that was never used for training, tuning or checkpoint choice. The unit of reporting is the mean reward of one day, so each run gives seven values, and the standard deviations quoted are sample standard deviations over seeds. The traffic forecast used as an input comes from the city-wide forecaster described in Section 7.2, which was trained without days 5 and 6.

To avoid a misleading comparison, the controllers are judged against several non-learning references: an equal split; an expert rule that sizes URLLC and mMTC capacity to a target utilisation and gives the remainder to eMBB, used with hand-set targets (untuned) and with the two targets chosen by grid search on the training days (tuned), each with current demand or with the forecast; and the same rule given the true next-interval demand, which is an upper bound for that rule.

## 7.2 Forecasting the whole city

The forecaster is the TGNN of Chapter 5, retrained properly. It sees the recent history of each cell, the cell's demand at the same time on the previous day, the time of day and a weekend flag, uses a residual output head without an upper limit, and is trained on random 32 by 32 patches of the 100 by 100 grid with each cell's traffic divided by its own typical level. Table 7.1 compares it with an earlier version of the same model trained on only five days, on every one of the 10,000 cells in the held-out week.

**Table 7.1:** Improvement over repeating the last value, by region (eMBB / URLLC / mMTC)

| Cells | Trained on 5 days | Trained on 39 days |
|---|---|---|
| 100 block cells | -12% / +17% / +21% | +21% / +36% / +35% |
| 300 cells around the block | -49% / +8% / +7% | +15% / +33% / +32% |
| 9,600 outer cells | -34% / +9% / +5% | +18% / +34% / +31% |
| All 10,000 cells | -34% / +9% / +7% | +18% / +33% / +31% |

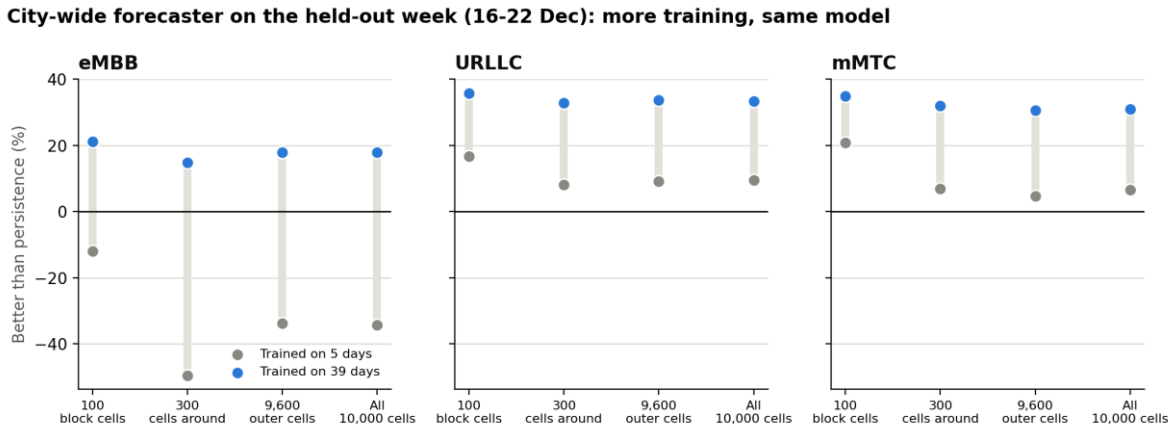



**Figure 7.1:** City-wide forecaster on the held-out week: five days of training against 39 days

Training length was the main problem. The five-day model is worse than persistence on eMBB in this week, while the 39-day model beats it in every region and slice. A first forecaster trained only on the 100 central cells over-predicted the quiet outer cells by about 6 Mbps and had 10.8 times the error of persistence there, which is why the city version uses per-cell scaling and patch training. Seasonal baselines are much worse than persistence at a ten-minute horizon: using the same time on the previous day gives roughly seven to eight times persistence's eMBB error in these runs (4.7 times in an independent check on a subsample of the raw data), so persistence is the baseline to beat.

## 7.3 Value of the forecast to the expert rules

**Table 7.2:** Reward of the expert rule on the held-out week

| Cells | Current demand | TGNN forecast | True next demand | Gain from the forecast (95% CI) |
|---|---|---|---|---|
| 9 | 0.794 | 0.825 | 0.945 | +0.030 ±0.008 (7 of 7 days) |
| 25 | 0.775 | 0.804 | 0.920 | +0.029 ±0.006 (7 of 7 days) |
| 49 | 0.758 | 0.788 | 0.902 | +0.029 ±0.004 (7 of 7 days) |
| 100 | 0.765 | 0.794 | 0.908 | +0.029 ±0.004 (7 of 7 days) |

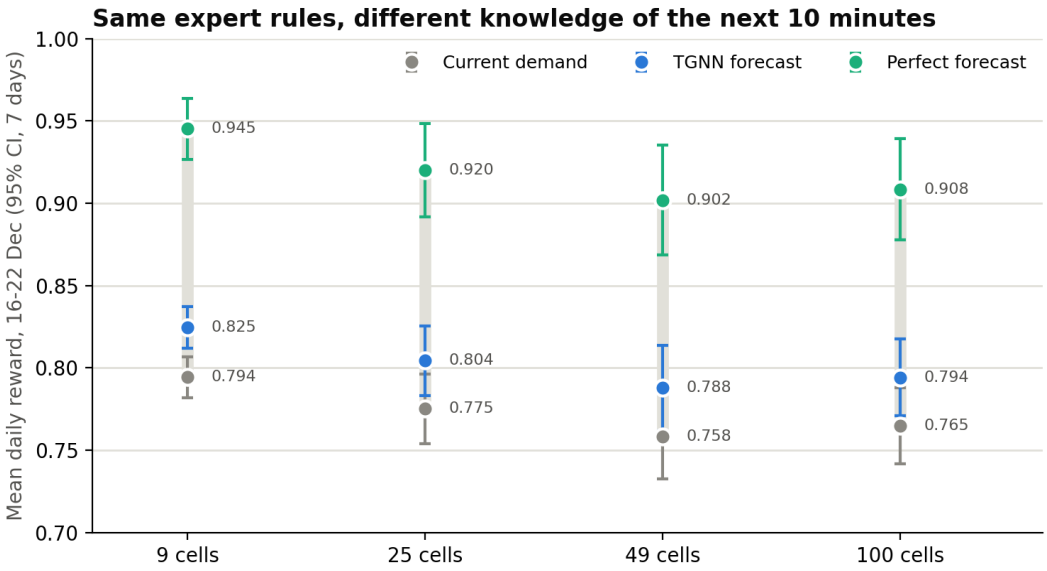



**Figure 7.2:** Reward of the expert rule with different knowledge of the next ten minutes

The forecast improves the expert rule by about 0.03 reward at every network size, on all seven test days, which recovers roughly 20% of the gap to knowing the true next demand. Using "same time yesterday" as the forecast is worse than using current demand (0.758 against 0.775 at 25 cells). The first study, on two test days with a forecaster trained on the 100 central cells, had suggested a gain of 0.04 to 0.05; the longer test week gives the smaller and better-supported figure.

## 7.4 MARL compared with a single centralised agent

The single agent observes every cell and outputs the slice shares of every cell, so its input and output grow with the network. The MARL controller uses one actor shared by all cells, each acting on its own seventeen local features, with a critic that also sees the mean of its neighbours' observations and actions and a reward that mixes each cell's reward with its neighbours'. Both learners receive the same data and the same 1,000 episodes of six hours, and the single agent is run with two actor learning rates (1e-4 and 3e-4), reporting the better. Table 7.3 gives the held-out week; 100 cells has one seed, the others two.

**Table 7.3:** Reward on the held-out week by network size

| Cells | MARL | Single agent (best rate) | Equal split | Expert, untuned, forecast | Expert, tuned, forecast | Actor parameters, MARL / single |
|---|---|---|---|---|---|---|
| 9 | 0.926 (0.929, 0.923) | 0.442 | 0.785 | 0.825 | 0.909 | 19,332 / 114,468 |
| 25 | 0.833 (0.894, 0.773) | 0.337 | 0.750 | 0.804 | 0.877 | 19,332 / 200,548 |
| 100 | 0.857 (0.857) | 0.284 | 0.735 | 0.794 | 0.865 | 19,332 / 604,048 |

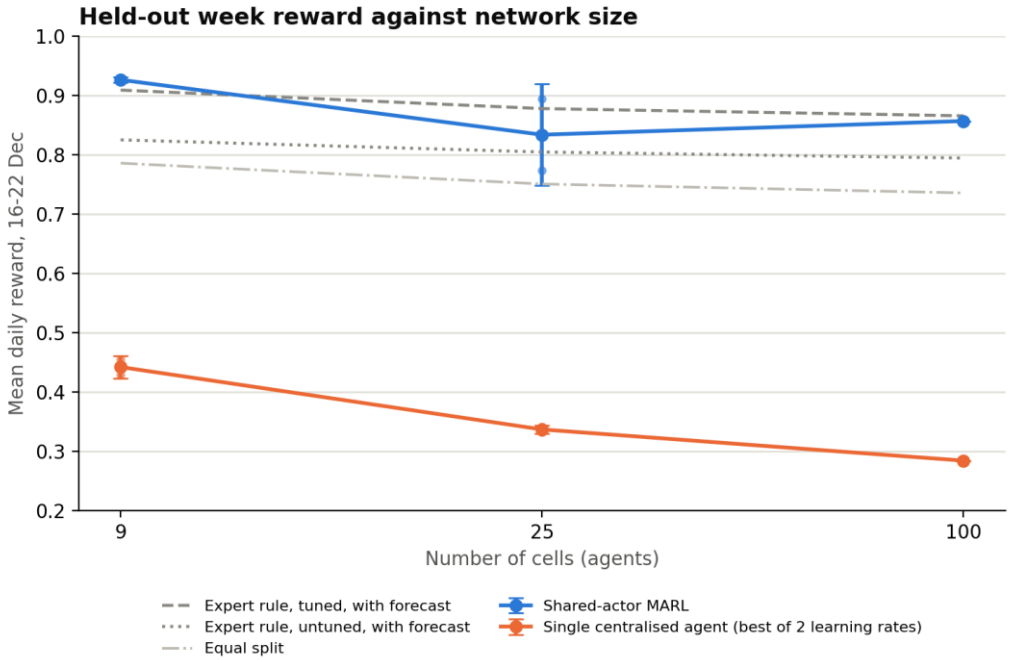



**Figure 7.3:** Held-out week reward against the number of cells

The single agent is far worse at every size and falls from 0.442 at 9 cells to 0.284 at 100 cells, below even the equal split. Both learning rates give about the same result, and its training reward barely changes between episodes 500 and 1,000 at 25 and 100 cells, so this is not a tuning accident. The shared actor keeps the same 19,332 parameters at every size because every cell's experience trains the same network, so a trained actor can be applied to a network of any size, and it matches how a radio access network is operated: each cell decides from its own and its neighbours' measurements, with no central controller and no single point of failure. With this short budget, however, MARL is only level with the tuned expert rule (better at 9 cells, worse at 25 and 100), and its two seeds at 25 cells disagree (0.894 and 0.773).

## 7.5 Training MARL long enough

The 1,000-episode runs were still improving. With 4,000 episodes and the checkpoint chosen on the validation days, three seeds at 25 cells reach 0.899, 0.899, 0.903 on the held-out week, and the average of the three actors reaches 0.904 (Table 7.4).

**Table 7.4:** Reward at 25 cells on the held-out week

| Controller | Reward |
|---|---|
| Equal split | 0.750 |
| Expert rule, untuned, current demand | 0.775 |
| Expert rule, untuned, with forecast | 0.804 |
| Expert rule, tuned, current demand | 0.847 |
| Expert rule, tuned, with forecast | 0.877 |
| MARL, 2,000 episodes, final checkpoint, 3 seeds | 0.881 (sd 0.021) |

| Controller | Reward |
|---|---|
| MARL, 4,000 episodes, validation-selected, 3 seeds | 0.899, 0.899, 0.903 |
| MARL, ensemble of those three actors | 0.904 |
| MARL with neighbour communication, 1 seed | 0.899 |
| MARL starting from the tuned expert, 1 seed | 0.894 |
| Expert rule on the true next demand (upper bound for the rule) | 0.920 |

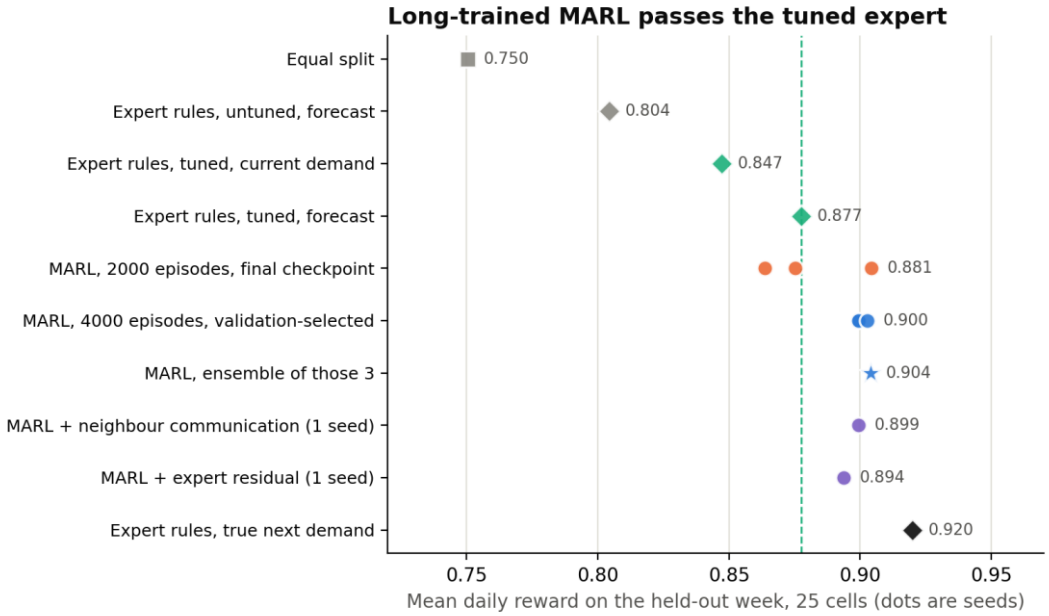



**Figure 7.4:** MARL and the reference controllers at 25 cells on the held-out week

The ensemble beats the tuned expert rule with the forecast by +0.026 reward (95% confidence interval ±0.005), higher on 7 of the 7 days, and each seed on its own gains between +0.022 and +0.025, also on all seven days. Against the tuned rule that uses current demand the gain is +0.057. Choosing the checkpoint on validation days is what makes this reliable: the final-checkpoint runs span 0.864 to 0.904, while the validation-selected runs lie between 0.899 and 0.903.

To see where the gain comes from, we compared one trained controller with the expert rule on the same week (Table 7.5). MARL reserves about twice as much capacity for URLLC and mMTC, so those slices meet their SLA almost every time, and it gives up a little eMBB service. This is a trade the reward asks for, found from the data rather than set by hand; the grid-searched expert rule finds a similar trade with two numbers, which is why it is the appropriate opponent.

**Table 7.5:** What the controllers do on the held-out week (25 cells)

| Controller | Share of capacity eMBB / URLLC / mMTC | SLA met eMBB / URLLC / mMTC | Demand served |
|---|---|---|---|
| MARL (seed 0, 2,000 episodes) | 68% / 23% / 9% | 68% / 99% / 100% | 92.4% |
| Expert rule, untuned, with forecast | 85% / 11% / 4% | 72% / 80% / 61% | 94.8% |
| Expert rule on true next demand | 85% / 11% / 4% | 72% / 100% / 100% | 94.6% |

Two additions that might have helped did not. Letting each actor see its neighbours' observations gave 0.899, starting from the tuned expert and learning a correction gave 0.894, and combining both gave 0.892 (one seed each), none better than plain MARL trained long (0.899 to 0.903). An actor loss that stays close to the expert rule (the TD3+BC form), run without the forecast, reached 0.844 over three seeds.

## 7.6 Does the forecast help MARL?

With the TGNN forecast as an input, MARL (2,000 episodes, final checkpoint, three seeds) scores 0.881 with a standard deviation of 0.021; without it, 0.851 with a standard deviation of 0.077. The difference of +0.030 is in the expected direction and of the same size as the forecast's gain for the expert rule, but the seeds overlap, so it is not established. The earlier result that the forecast did not help MARL (0.765 against 0.778) came from a run of only 300 episodes on five days and is superseded.

## 7.7 A change in the traffic mix

Finally, controllers trained or tuned on ordinary traffic were scored without any retraining on two altered traffic mixes, the 6G growth scenario (eMBB x1.3, URLLC x2, mMTC x3) and a massive-IoT scenario (mMTC x10), with the forecast scaled in the same way (Table 7.6).

**Table 7.6:** Reward on the held-out week when the traffic mix changes

| Traffic | Equal split | Tuned expert, forecast | MARL, forecast | MARL, no forecast |
|---|---|---|---|---|
| Recorded traffic | 0.750 | 0.877 | 0.881 (0.021) | 0.851 (0.077) |
| 6G growth mix | 0.615 | 0.762 | 0.724 (0.028) | 0.692 (0.062) |
| Massive-IoT mix | 0.666 | 0.725 | 0.672 (0.024) | 0.685 (0.015) |

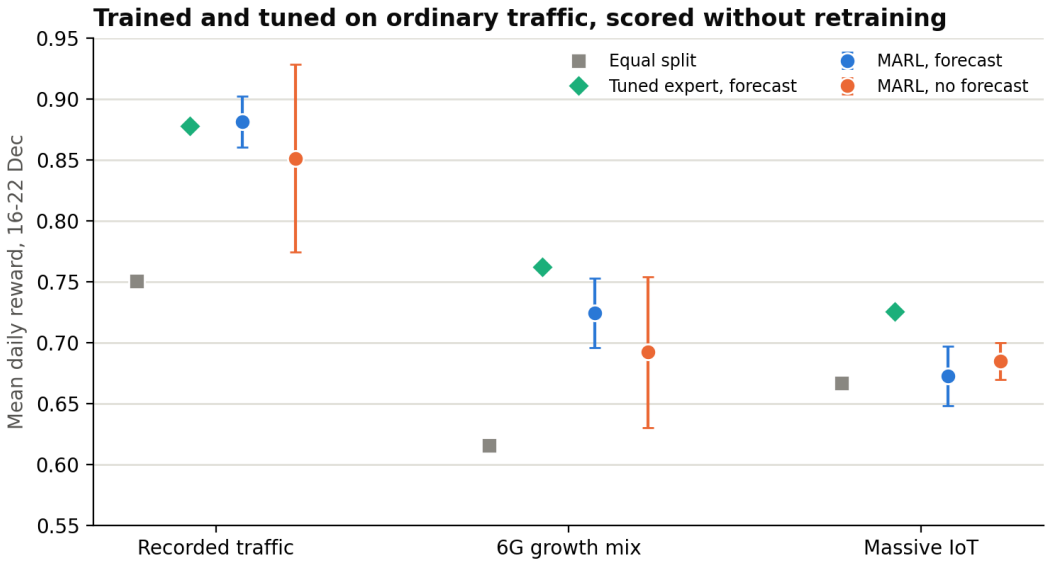



**Figure 7.5:** Controllers tuned on ordinary traffic and scored on altered traffic without retraining

The trained controllers lose to the hand-tuned rule once the mix changes. The case for MARL in this work is therefore that it finds a good policy from data without hand tuning and scales to any number of cells, not that it is robust to changes it was not trained on.

## 7.8 Self-supervised learning revisited

The first evaluation of the self-supervised encoder was flawed. 807,235 of the 1,215,890 flows in 5G-NIDD (66%) are exact duplicates, so identical flows appeared in both the training and the test set, and the row-index columns leak identity. After removing the duplicates (408,655 flows), dropping the identifier columns and using a stratified split over the nine attack classes, we tested where pretraining should help most, with few labelled flows. Two pretraining methods were used, masked-feature reconstruction and a contrastive method (SCARF), each followed by a linear classifier on the frozen embedding or by fine-tuning, and compared with the same classifiers on the raw features and with LightGBM (Table 7.7).

**Table 7.7:** 5G-NIDD attack classification, macro-F1 on 81,731 unseen flows

| Labelled flows | LightGBM, raw | MLP, raw | Embedding (masked), linear | Fine-tuned (contrastive) | Raw + embedding, linear |
|---|---|---|---|---|---|
| 50 | 0.643 | 0.694 | 0.627 | 0.688 | 0.690 |
| 200 | 0.852 | 0.858 | 0.763 | 0.850 | 0.848 |
| 1,000 | 0.905 | 0.895 | 0.859 | 0.894 | 0.888 |
| 5,000 | 0.919 | 0.911 | 0.901 | 0.901 | 0.915 |
| 20,000 | 0.932 | 0.924 | 0.913 | 0.925 | 0.924 |
| all | 0.936 | 0.925 | 0.912 | 0.929 | 0.926 |

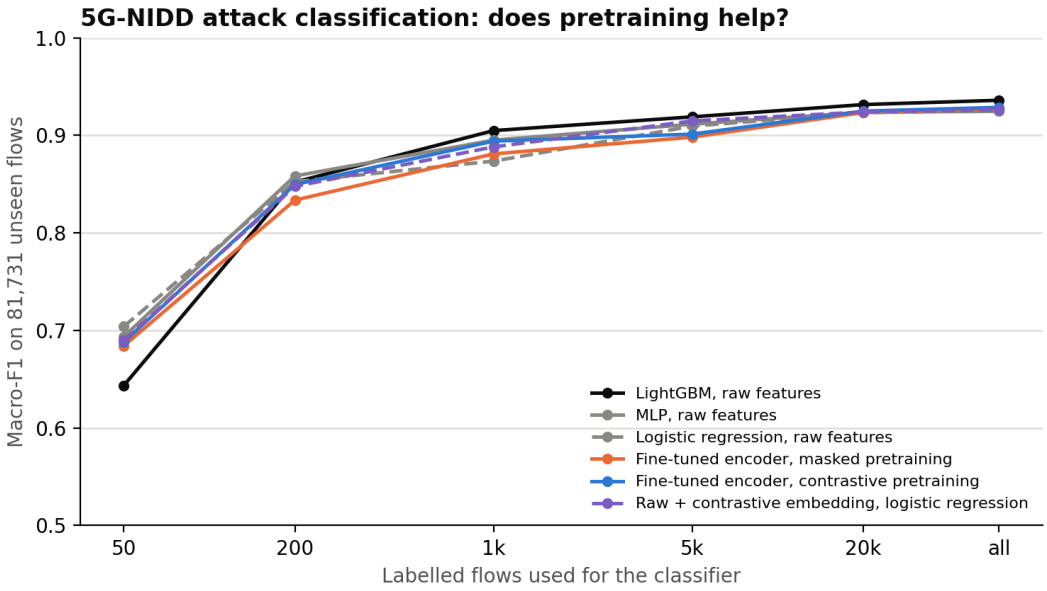



**Figure 7.6:** Macro-F1 against the number of labelled flows for raw features and pretrained encoders

Raw features alone reach a macro-F1 of 0.926 with logistic regression and 0.936 with LightGBM, against the 0.708 of the first evaluation, so most of the earlier gap came from the leakage. Pretraining never gives a reliable advantage over raw features: with 50 or 200 labels a raw-feature MLP or logistic regression beats the fine-tuned encoder, with all labels the fine-tuned contrastive encoder ties the same MLP (0.929 against 0.925) and stays below LightGBM, and adding the embedding to the raw features helps a linear model by at most 0.014, at 1,000 to 5,000 labels. We report this as a negative result.

## 7.9 Controllers across the whole city

The three controllers used in the prototype (static equal split, the untuned expert rule on current demand, and the same rule with the city-wide forecast) were run on all 10,000 cells for one held-out day with scripted incidents (Table 7.8). The forecast is made by the same forecaster, which never saw this day.

**Table 7.8:** Controllers on all 10,000 cells, Wednesday 6 November 2013

| Controller | SLA met (eMBB / URLLC / mMTC) | Reward, whole city | Reward, dense block | Reward, outer city |
|---|---|---|---|---|
| Static | 0.919 / 0.997 / 1.000 | 0.974 | 0.709 | 0.977 |
| Reactive | 0.975 / 0.711 / 0.568 | 0.839 | 0.735 | 0.840 |
| Predictive | 0.975 / 0.786 / 0.601 | 0.868 | 0.773 | 0.869 |

In the dense block the predictive controller does best, but across the city as a whole the static split does best. Most of the city is lightly loaded, and the untuned expert rule gives URLLC and mMTC almost no capacity when their traffic is near zero, so these slices miss their SLA when traffic returns.

A rule tuned on training days (Section 7.5) would reserve more capacity for these slices; the monitor uses the untuned rule as the reactive baseline.

# Chapter 8
# The Digital Twin Dataset

## 8.1 Overview

Using the unified environment we generated a long dataset in which every row describes one cell in one 10-minute interval. It is produced by src/unified/twin_dataset.py; the full data is stored as Parquet files on the project drive, and summaries and samples are kept in the repository under reports/unified_twin.

**Table 8.1:** Size of the Digital Twin dataset

| Build | Cells | Days | Scenarios | Rows | Size |
|---|---|---|---|---|---|
| Core year | 400 (central 20 x 20 block) | 364 | 6 | 125,796,000 | 30.8 GB |
| Full city | 10,000 (whole grid) | 62 | 3 | 267,810,000 | 40.5 GB |
| Total | | | | 393,606,000 | |

The traffic for the 62 recorded days is real. The remaining days of the core year are built from real days of the same weekday, with a growth trend of 20% per year and small random variation, and every row records whether it is real or bootstrapped. Latency and jitter follow the calibrated curves, radio fields use the 3GPP TS 38.214 CQI table, and power uses the EARTH base-station model. The slice mapping, the industrial zone, per-user rates and packet sizes are assumptions. Each row has 122 columns covering calendar, control, events, radio and capacity, neighbourhood, energy, per-slice demand and QoS, reward and noisy telemetry; the full list is in Appendix B.

## 8.2 Scenarios

All scenarios use the same days, behaviour policies and random noise, so that any difference between two scenarios is caused only by the scenario itself. Table 8.2 gives the results for the core year.

**Table 8.2:** Scenarios of the core year

| Scenario | What changes | SLA met (eMBB / URLLC / mMTC) | Reward | Energy (MWh) |
|---|---|---|---|---|
| baseline | Recorded traffic without incidents | 0.704 / 0.731 / 0.651 | 0.763 | 3,473 |
| events | Adds flash crowds, outages, interference, URLLC bursts, mMTC storms and capacity degradation | 0.703 / 0.730 / 0.651 | 0.763 | 3,473 |
| growth_6g | eMBB x1.3, URLLC x2 and mMTC x3 | 0.553 / 0.696 / 0.610 | 0.688 | 3,648 |
| energy_saving | Lightly loaded cells sleep from 01:00 to 05:00 and hand their traffic to neighbours | 0.703 / 0.737 / 0.659 | 0.766 | 3,435 |
| industrial_urllc | Industrial cells see URLLC x4 during weekday shifts | 0.693 / 0.723 / 0.646 | 0.756 | 3,481 |
| massive_iot | mMTC x10, with a further x3 at the start of every hour | 0.554 / 0.710 / 0.524 | 0.660 | 3,628 |

**Table 8.3:** Effect of each injected event (core year, events scenario)

| Event | Rows | SLA met (eMBB / URLLC / mMTC) | Reward |
|---|---|---|---|
| none | 20,932,659 | 0.704 / 0.730 / 0.651 | 0.763 |
| flash crowd | 19,740 | 0.217 / 0.683 / 0.598 | 0.532 |
| cell outage | 985 | 0.000 / 0.095 / 0.019 | 0.019 |
| interference spike | 8,883 | 0.577 / 0.697 / 0.629 | 0.704 |
| urllc burst | 814 | 0.608 / 0.510 / 0.575 | 0.631 |
| mmtc storm | 1,966 | 0.658 / 0.730 / 0.344 | 0.679 |
| capacity degradation | 953 | 0.323 / 0.710 / 0.647 | 0.578 |

Each type of event affects the slices it should: flash crowds and capacity degradation reduce eMBB compliance, URLLC bursts reduce URLLC compliance, mMTC storms reduce mMTC compliance, and outages affect every slice.

## 8.3 Energy saving across the city

**Table 8.4:** Night-time sleep across the city (01:00 to 05:00, 62 days)

| Measure | Baseline | Energy saving |
|---|---|---|
| Night-time energy (MWh) | 1983 | 1593 (19.6% lower) |
| Night cell-intervals asleep | 0% | 47.6% |
| SLA met on awake cells (eMBB / URLLC / mMTC) | 0.983 / 0.761 / 0.724 | 0.976 / 0.720 / 0.693 |
| Mean load of awake cells | 0.045 | 0.066 |

Putting quiet cells to sleep at night saves about a fifth of night-time energy, or 3.2% over the whole 62 days. The cost falls on the cells that stay awake and absorb the extra traffic, where URLLC compliance drops by about four percentage points. Sleeping cells carry no traffic and so meet their SLA trivially, which is why the comparison above is restricted to awake cells.

## 8.4 Checks on the generated data

A three-day test build of all six scenarios was checked before the full run. The scenarios contain identical rows of interval and cell; traffic is conserved under energy saving (the traffic handed over equals the traffic received); the industrial and massive-IoT multipliers are applied exactly; 1.0% of telemetry is missing, as designed; and the forecasting targets for 10 and 60 minutes ahead equal the cell's demand one and six rows later. These checks are also covered by automated tests in the repository.

# Chapter 9
# Prototype: the Slice Twin Monitor

## 9.1 Purpose

The School policy requires each B.Tech project to end in a working product or prototype. Ours is Slice Twin, a browser-based monitor for watching how different controllers divide each cell's bandwidth between slices during one held-out day of real Milan traffic with scripted incidents. Its interaction model is borrowed from Cisco Packet Tracer, with a real-time mode, a simulation mode that stops at each new event, an event list and step-by-step playback. The source is in app/slice_twin of the repository.

## 9.2 Views

**Block view.** The 100 central cells are drawn as rings whose segments show each cell's split between slices, with moving packets indicating served traffic. Selecting a cell shows its demand, allocation, latency against target and SLA status for each slice.

**City view.** All 10,000 grid cells are drawn as a map coloured by real traffic, cell load or SLA misses, with landmarks (Duomo, Milano Centrale, San Siro, Linate airport, Navigli and Citta Studi) and the active incidents. Selecting a cell compares the three controllers on that cell over the day, and a cell inside the central block can be opened in the block view.

**Controllers.** Static (equal thirds), Reactive (the expert policy on current demand) and Predictive (the same policy with the TGNN forecast).

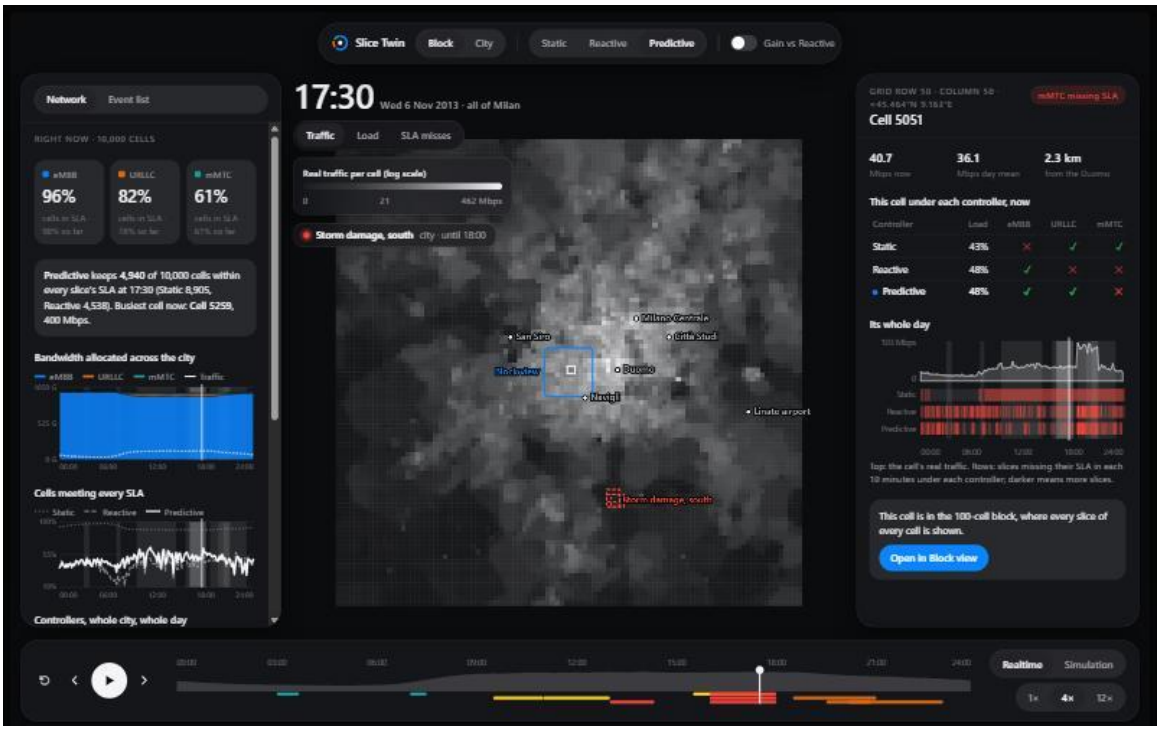



**Figure 9.1:** City view coloured by real traffic (dark theme)

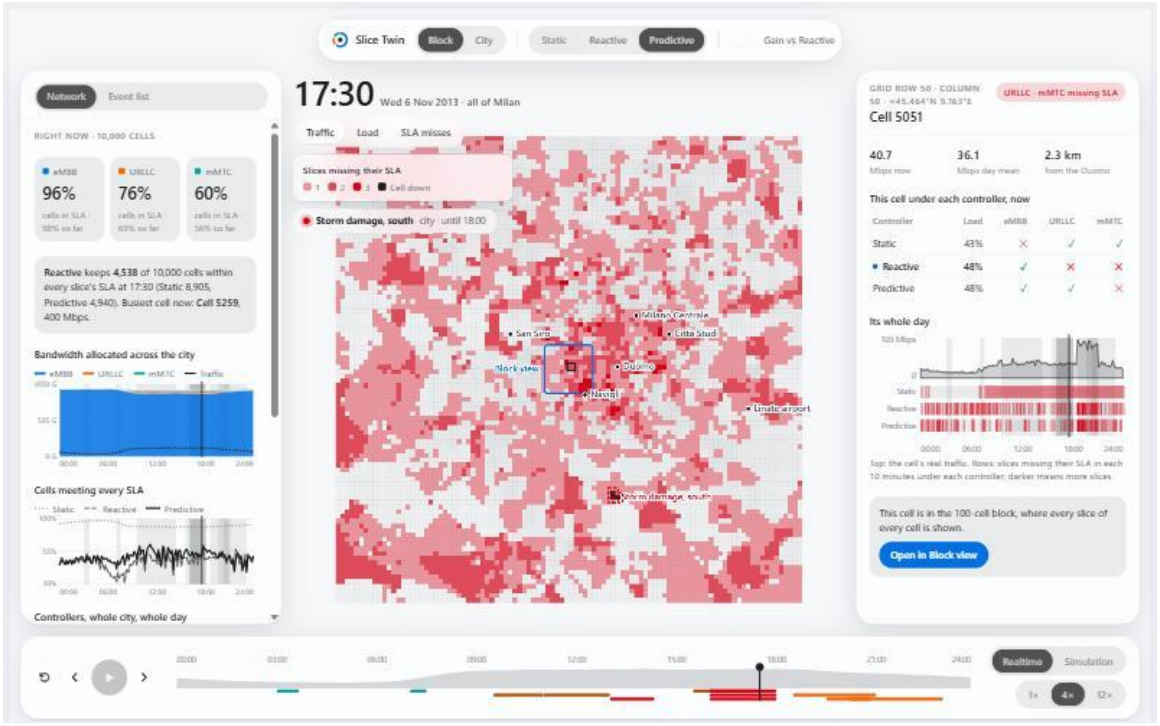



**Figure 9.2:** City view coloured by SLA misses under the reactive controller (light theme)

# Chapter 10
# Limitations of the Current Evidence

Before planning publications we checked every claim against the stored result files, and re-ran the weakest ones with more data and longer training. The results that hold up best are the city-wide forecaster, the value of the forecast to the expert rules, and the advantage of shared-parameter MARL over a single centralised agent. The following limitations remain.

## 10.1 Statistical strength

Every result in Chapter 7 is measured on a held-out week with one reward per day, and the paired gains of the forecast and of the long-trained MARL over the tuned expert rule are positive on all seven days with confidence intervals that exclude zero. The days are consecutive and therefore not independent, so the intervals are indicative. The learned controllers use two or three seeds (one seed for the 100-cell scaling point), well below the ten seeds that are now standard in reinforcement learning papers, and the final-checkpoint results at 25 cells vary a lot between seeds (0.77 to 0.90).

## 10.2 Scope of the MARL result

The finding that MARL beats a tuned expert rule comes from 25 cells, one week and one region, with the long training protocol run only at that size. At 9 and 100 cells MARL was trained with the shorter protocol, where it is level with the tuned rule. The expert baseline has only two tuned numbers, so a richer tuned baseline, with targets per hour or per cell, might close part of the gap. Standard multi-agent baselines such as independent agents, IPPO and MAPPO have not been run. When the traffic mix changes without retraining, MARL is worse than the tuned rule.

## 10.3 Model fidelity

QoS comes from a calibrated surrogate model rather than a packet-level simulator, and the reference telemetry used for calibration is itself simulated. The reinforcement learning experiments use the deterministic version of the latency model. A load sweep in Simu5G would provide an independent check of the latency curve. The slice mapping, the capacity and the reward weights are assumptions, and rankings of controllers may depend on them: the URLLC and mMTC slices carry most of the reward weight.

## 10.4 Negative results

Self-supervised pretraining gives no reliable advantage over raw features on 5G-NIDD at any label budget, and the same was found earlier for the Milan forecast. Trained controllers lose to the tuned

rule when the traffic mix changes, the static split outperforms the predictive controller on quiet cells in the city-wide demonstration, and the effect of the forecast on MARL, although in the expected direction, is not statistically established.

## 10.5 The Digital Twin claim

The approve or reject gate exists only in the early B5G orchestrator, where it approves no actions, and the unified environment has no gate. Describing the controllers as validated inside a Digital Twin is therefore not yet justified. A one-step look-ahead gate in the unified environment, which rejects actions whose predicted SLA falls below a threshold and measures the effect on reward, would make this claim testable at modest cost.

# Chapter 11
# Publication Plan and Pending Work

The plan below is based on a search of existing surveys and closely related original work from 2021 to 2026, a review of suitable venues, and the limitations in Chapter 10. It is proposed for the guide's approval before drafting begins.

## 11.1 Review paper

**Proposed title.** Predict-then-Allocate for AI-Native 6G Network Slicing: A Systematic Review of Forecasting, Digital Twins and Multi-Agent Reinforcement Learning, with a Dataset and Reproducibility Audit.

Recent surveys cover each building block separately, including Digital Twin networks, graph-based traffic forecasting, reinforcement learning for slicing and foundation models for wireless networks. We did not find a survey that examines how forecasts are connected to the allocation decision, the different roles a Digital Twin can play in that loop, or which datasets and simulators support each stage of the slicing pipeline. The closest competitors are a 2026 review of MARL for slicing in PeerJ Computer Science and several survey preprints from mid-2026, which must be read in full before the paper claims novelty.

The review would follow the PRISMA 2020 method over IEEE Xplore, Scopus, the ACM Digital Library and arXiv for 2021 to 2026, with two reviewers screening each paper. The proposed taxonomy has five layers (representation, prediction, Digital Twin, allocation, and the coupling between prediction and control) and two cross-cutting dimensions (data provenance and reproducibility). A narrower fallback is a review of datasets, simulators and reproducibility for AI-native slicing. The suggested target is IEEE Access, whose article processing charge is US$2,160, or Elsevier Computer Networks on its subscription route if the charge cannot be funded; a realistic submission date is the end of February 2027.

## 11.2 Original paper

**Proposed title.** How Much Is Traffic Prediction Worth for Multi-Cell Network Slicing? A Real-Traffic, City-Scale Benchmark.

The contributions would be the open multi-cell slicing benchmark on real Milan traffic, the city-scale forecasting study (the failure of the model trained on the central cells and the success of the patch-trained, per-cell-scaled model), the value of the forecast measured on a held-out week, and a comparison of a parameter-shared MARL controller with a single centralised agent and with a grid-

tuned expert rule, with the negative findings reported. The terms Digital Twin and SSL should be kept out of the paper title, while the project title remains appropriate for the thesis.

Several of the checks that reviewers will ask for are already done in Chapter 7: sample standard deviations and paired confidence intervals over seven held-out days, a tuned expert baseline, a seasonal forecasting baseline and a 39-day training set. Still to do are ten seeds, IPPO and MAPPO baselines, the long training protocol at 9 and 100 cells, a richer tuned expert baseline, a Simu5G check of the latency model, and an ablation of per-cell scaling against patch training in the forecaster. Suitable first venues are an IEEE ICC 2027 workshop, whose deadline is expected in January 2027, or IEEE CONECCT 2027, with a paper deadline of 15 February 2027. A journal extension with the Simu5G check and a public release of the dataset could follow. Two arXiv preprints (2306.09290 and 2605.03345) may already use the Milan data for slicing and must be checked first.

## 11.3 Questions for the guide

1. Does the guide approve the proposed review topic and the framing of the original paper?
2. Do the 5 marks for a paper apply to this semester, and is a draft ready for submission sufficient?
3. Can the department or library support the IEEE Access article processing charge, or should we target journals without a charge?
4. Should the Digital Twin gate and the Simu5G validation be completed before the final review, or reported as future work?

## 11.4 Pending work

**Table 11.1:** Pending work

| Item | Responsible | Target |
|---|---|---|
| Final review presentation | All members | Before the final review |
| Thesis in the department template; signed soft-bound copy with silver title page | All members | At the final presentation |
| Message to the guide with this record and the publication plan | Keerthivasan | 20 to 22 October 2026 |
| Ten seeds, IPPO and MAPPO baselines, long protocol at 9 and 100 cells | Keerthivasan and Krish | November to December 2026 |
| Ablation of per-cell scaling and patch training in the forecaster | Thrishala and Keerthivasan | November 2026 |
| Write up the SSL negative result with the label-efficiency study | Sriranjana | November 2026 |
| Literature search, screening and coding for the review | All members | November 2026 to January 2027 |

| Item | Responsible | Target |
|---|---|---|
| First drafts of both papers | All members | After approval, December 2026 to February 2027 |
| Digital Twin gate and Simu5G load sweep | Keerthivasan | Optional, December 2026 |

# References


[1] G. Barlacchi et al., "A multi-source dataset of urban life in the city of Milan and the Province of Trentino," Scientific Data, vol. 2, 150055, 2015.

[2] H. Samarakoon et al., "5G-NIDD: A comprehensive network intrusion detection dataset generated over 5G wireless network," arXiv:2212.01298, 2022.

[3] D. Bega, M. Gramaglia, M. Fiore, A. Banchs and X. Costa-Perez, "DeepCog: Optimizing resource provisioning in network slicing with AI-based capacity forecasting," IEEE Journal on Selected Areas in Communications, vol. 38, no. 2, pp. 361-376, 2020.

[4] W. Hamilton, Z. Ying and J. Leskovec, "Inductive representation learning on large graphs," in Advances in Neural Information Processing Systems, 2017.

[5] R. Lowe et al., "Multi-agent actor-critic for mixed cooperative-competitive environments," in Advances in Neural Information Processing Systems, 2017.

[6] Y. Yang et al., "Mean field multi-agent reinforcement learning," in Proc. International Conference on Machine Learning, 2018.

[7] T. Kim et al., "Reversible instance normalization for accurate time-series forecasting against distribution shift," in Proc. International Conference on Learning Representations, 2022.

[8] G. Auer et al., "How much energy is needed to run a wireless network?," IEEE Wireless Communications, vol. 18, no. 5, pp. 40-49, 2011.

[9] 3GPP, "NR; Physical layer procedures for data," TS 38.214.

[10] G. Nardini, D. Sabella, G. Stea, P. Thakkar and A. Virdis, "Simu5G: An OMNeT++ library for end-to-end performance evaluation of 5G networks," IEEE Access, vol. 8, pp. 181176-181191, 2020.

[11] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in Advances in Neural Information Processing Systems, 2021.

[12] M. J. Page et al., "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," BMJ, vol. 372, n71, 2021.

# Appendix A: Literature Review Notes

*These notes are reproduced from the team's literature review document with light formatting changes.*

| **Paper 1: AI-Driven Digital Twins: Optimizing 5G/6G Network Slicing with NTNs** | |
|---|---|
| Ali & Arslan (2025) — IEEE Wireless Communications Letters | |
| Problem Statement | Static Digital Twin updates can't keep pace with the rapid topology changes caused by UAV-mounted flying base stations during disaster recovery or urban blockage events, leading to latency spikes for NTN-integrated eMBB slices. |
| Methodology / Algorithm | A real-time DT synchronization scheme continuously tracks the spatial distance between mobile users and the flying base station, feeding a continuous-action DDPG agent that allocates bandwidth. The state space folds in path loss, Rayleigh fading and satellite interference. |
| Dataset / Simulation Tool | Custom Python-based 5G simulator with 50 UEs over 20 MHz bandwidth. |
| Results | ~25% latency reduction vs static allocation (avg. 33 ms), jitter down to 2.1 ms, ~92% resource utilization. |
| Research Gap | Assumes clean, noise-free telemetry and has no predictive topology mapping — it reacts to raw signals rather than filtering anomalies through learned representations. |
| How it relates to our project | Confirms DDPG works well inside a Digital Twin loop, but its lack of representation learning is exactly the gap our SSL module is meant to close, and its purely reactive design motivates adding TGNN-based prediction. |

| **Paper 2: A Flexible and Scalable Multi-Agent Learning Framework for Dynamic RAN Slicing in 6G Native-AI Networks** | |
|---|---|
| Li, Wang, et al. (2026) — IEEE Transactions on Mobile Computing | |
| Problem Statement | Changing slice configurations forces costly model retraining, and frequent radio-resource reconfiguration creates heavy signalling and computational overhead. |
| Methodology / Algorithm | Proposes a Statistic-Field Deep Truncated Monte Carlo (SF-DTMC) algorithm operating in a RAN Partially Observable Stochastic Game (RAN-POSG), paired with an Adaptive Retention Framework (ARF) that penalizes unnecessary state transitions to balance responsiveness against reconfiguration stability. |
| Dataset / Simulation Tool | NS-3-based high-fidelity RAN slicing simulator. |
| Results | Large drop in reconfiguration time and SLA violations compared with LLM-based agents and standard RL baselines. |
| Research Gap | Scalability is limited once you move past tightly localized cell clusters, and control decisions still rely purely on local/reactive state observations. |
| How it relates to our project | Backs our decentralized multi-agent design, but our framework goes further by adding SSL and TGNN to enrich state representation and add predictive foresight, which this paper doesn't attempt. |

| **Paper 3: A Framework for AI-Native Semantic-Based Dynamic Slicing for 6G Networks** | |
|---|---|
| Chowdhury et al. (2025) — TechRxiv | |
| Problem Statement | Traditional architectures treat data content and network resource orchestration as separate concerns — insufficient for 6G's semantic-aware ambitions. |
| Methodology / Algorithm | Introduces a semantic slicing framework built around a horizontal "Reasoning Plane" that sits between the data and control planes, using symbolic AI, GFlowNets, and signaling games to build shared semantic languages. |
| Dataset / Simulation Tool | Python / TensorFlow. |
| Results | Faster semantic task execution and improved convergence as the system settles on a shared semantic vocabulary. |
| Research Gap | Highly theoretical — it doesn't quantify the computational latency the Reasoning Plane would add on real edge hardware. |
| How it relates to our project | Supplies the conceptual blueprint for the Reasoning Plane and data slicing, which we position as future work once the core SSL+TGNN+MARL pipeline is validated. |

| **Paper 4: GAN-Enhanced Deep Reinforcement Learning for Semantic-Aware Resource Allocation in 6G Network Slicing** | |
|---|---|
| Daniel Benniah John (2026) — arXiv (cs.NI) | |
| Problem Statement | Discrete action quantization plus "semantic blindness" in classical resource allocation wastes an estimated 35% of bandwidth on redundant data. |
| Methodology / Algorithm | GAN-DDPG framework: conditional GANs generate synthetic semantic traffic while a continuous-action DDPG agent optimizes allocation against a semantic-aware reward instead of raw bit-rate maximization. |
| Dataset / Simulation Tool | Python / custom 6G slicing simulator. |
| Results | +22% URLLC spectral efficiency, +20% eMBB spectral efficiency, -31% packet loss, -18% latency versus baseline. |
| Research Gap | Semantic state definitions are domain-specific and don't generalize; semantic extraction itself adds computational delay at the edge. |
| How it relates to our project | Gives concrete empirical proof that reward-level semantic awareness beats classical MARL objectives — direct evidence for why we flag semantic communication as a strong future-work extension. |

| **Paper 5: Multi-Agent Reinforcement Learning for Slicing Resource Allocation in Vehicular Networks** | |
|---|---|
| (vehicular networks MARL study) (2023) — IEEE Transactions on Intelligent Transportation Systems | |
| Problem Statement | High-speed vehicular mobility forces frequent RAN slicing changes, generating heavy reconfiguration overhead. |
| Methodology / Algorithm | Formulates the problem as a POMDP and solves it with Multi-Agent DDPG (MADDPG) to minimize long-term system cost. |
| Dataset / Simulation Tool | Python with a Mininet emulator and Ryu SDN controller. |

| | |
|---|---|
| Results | Lower system cost overall, with an average QoS satisfaction rate of 96.5%. |
| Research Gap | Validation stays confined to simulation (true of ~86% of surveyed studies), and coordination delays of 120–180 ms exceed URLLC latency budgets. |
| How it relates to our project | Validates multi-agent handling of mobility, but has no Digital Twin safety net for policy exploration — the gap our NDT sandbox is designed to fill. |

**Paper 1: Native Network Digital Twin Architecture for 6G: From Design to Practice (2025)**

***1. Problem Statement***

- Existing Network Digital Twins (NDTs) lack standardized architecture, real-time synchronization, and efficient data collection for large-scale 6G networks.
- Traditional KPI-based monitoring cannot support service validation, prediction, or safe testing of network changes.
- The paper proposes a native Network Digital Twin architecture for real-time monitoring, simulation, verification, and optimization.

***2. Methodology / Algorithm***

- Introduces a three-layer architecture: Network Element Layer, NDT Layer, and Application Layer.
- Defines the complete NDT lifecycle: Preparation, Creation, Runtime, and Feedback.
- Proposes native high-speed data collection using kernel pass, sidecar agents, Kafka/RDMA/QUIC.
- Uses a gray-box modelling approach combining protocol knowledge with learned behaviour.

***3. Dataset / Simulation Tool***

- No public dataset is used.
- Validated using a real operator-inspired signalling storm scenario during disaster recovery.
- Digital Twin is built using real topology, signalling statistics, historical traffic patterns, and network behaviour models.

***4. Results***

***Achieved ≈3% MAPE for registration arrival rate prediction.***

***Achieved ≈25% MAPE for successful registration count.***

- Successfully replicated signalling storm behaviour and validated safe traffic control strategies.

- Demonstrated the effectiveness of the proposed architecture and gray-box modelling.

***5. Research Gap***

- Evaluation is limited to a signalling storm use case.
- No Graph Neural Networks (GNN), Temporal GNN (TGNN), Self-Supervised Learning (SSL), or Reinforcement Learning (RL) are incorporated.
- Focuses on simulation and verification rather than autonomous optimization.

***6. How it Relates to Our Project***

- Provides the architectural foundation for our Digital Twin framework.
- Supports real-time synchronization, modular Digital Twin lifecycle, and gray-box modelling.
- Our work extends this architecture using SSL, TGNN, and MARL for intelligent network slicing.

**Paper 2: Attention to Virtualization: Making Network Digital Twins Aware of Network Slicing (2025)**

***1. Problem Statement***

- Existing Digital Twins ignore virtualization effects such as resource contention between virtual network functions.
- Conventional GNNs suffer from over-smoothing and poor long-term dependency modelling.
- The paper proposes a virtualization-aware Digital Twin using Graph Attention Networks for improved KPI prediction.

***2. Methodology / Algorithm***

- Measures virtualization overhead using Docker-based UPF deployments.
- Represents physical and virtual resources as a graph.
- Combines GraphSAGE + Graph Attention Network (GAT) to learn virtualization-aware node embeddings.
- Performs "what-if" analysis before slice deployment using the Digital Twin.

***3. Dataset / Simulation Tool***

- Experimental measurements collected from Docker-based UPFs.

***Digital Twin evaluated using an extended OMNeT++ simulator.***

- Simulation includes 30 CNFs, 5 physical nodes, and GraphSAGE + GAT trained using a 70/30 split.

***4. Results***

- Virtualization significantly increases delay and packet loss.
- GraphSAGE + GAT outperforms conventional GNNs in KPI prediction.
- Achieves lower MAE and MSE than RouteNet-Fermi.
- Improves delay prediction across eMBB, URLLC, and mMTC slices.

***5. Research Gap***

- Uses a static graph and cannot model temporal traffic evolution.
- Depends on supervised learning with labelled data.
- No Reinforcement Learning for autonomous optimization.

***6. How it Relates to Our Project***

- Introduces virtualization-aware Digital Twins for network slicing.
- Provides the graph abstraction used in our framework

**Paper 3: A Graph Neural Network-Based Digital Twin for Network Slicing Management (2022)**

***1. Problem Statement***

- Existing analytical models and conventional deep learning methods cannot effectively model graph-structured communication networks.
- Shared physical resources create complex interactions between network slices, making E2E latency prediction difficult.
- The paper proposes a GraphSAGE-based Digital Twin for accurate End-to-End latency prediction.

***2. Methodology / Algorithm***

- Represents physical infrastructure and network slices as graphs.
- Uses GraphSAGE with neighbour sampling and mean aggregation to generate node embeddings.
- Aggregates node embeddings into slice embeddings and predicts End-to-End latency.

***Trains using the Log-Cosh loss and Adam optimizer.***

***3. Dataset / Simulation Tool***

- Uses three network topologies: NSFNET (14 nodes), GEANT2 (24 nodes), and Synthetic (50 nodes).

***Trains on 70% of graph nodes and evaluates on unseen nodes.***

- Uses traffic matrices, VNF resource utilization, and E2E latency as input features.

***4. Results***

- Achieved less than 5% prediction error for End-to-End latency.
- Successfully generalized to unseen network topologies.
- Demonstrated transfer learning for jitter prediction.
- Supported SLA monitoring, link failure recovery, and deployment optimization.

***5. Research Gap***

- Models the network as a static graph, ignoring temporal changes.
- Requires labelled datasets due to supervised learning.
- Does not perform Reinforcement Learning-based optimization.
- Does not explicitly model virtualization effects.

***6. How it Relates to Our Project***

- Provides the graph-based modelling foundation for our Digital Twin.
- Our work extends GraphSAGE using Temporal Graph Neural Networks (TGNNs).

**Paper 4: Revolutionizing Wireless Networks with Self-Supervised Learning: A Pathway to Intelligent Communications (2024)**

***1. Problem Statement***

- Supervised learning requires large labelled datasets, which are difficult to obtain in dynamic wireless communication networks.
- Massive amounts of unlabelled traffic, channel, and mobility data remain underutilized.
- The paper investigates Self-Supervised Learning (SSL) as an alternative learning paradigm for AI-native 6G networks.

***2. Methodology / Algorithm***

- Reviews three SSL paradigms: Generative, Contrastive, and Predictive learning.
- Describes the SSL workflow consisting of pretext task generation, representation learning, and downstream task fine-tuning.
- Surveys SSL applications in traffic prediction, channel estimation, anomaly detection, semantic communication, and network optimization.

*3. Dataset / Simulation Tool*

- Survey paper; no single dataset is used.
- Reviews multiple wireless communication datasets and existing SSL-based communication systems.
- Covers applications in wireless networking, semantic communications, and edge intelligence.

*4. Results*

- Demonstrates that SSL effectively learns meaningful representations from unlabelled wireless data.
- Reduces dependence on labelled datasets while improving model generalization.
- Successfully supports various communication tasks, including channel estimation, anomaly detection, and intelligent network management.

*5. Research Gap*

- Designing efficient pretext tasks for wireless communication remains challenging.
- Existing SSL methods have high computational complexity and limited robustness in dynamic environments.
- Integration with Digital Twins, TGNNs, and Reinforcement Learning remains largely unexplored.

*6. How it Relates to Our Project*

- Provides the motivation for using Self-Supervised Learning in AI-native 6G networks.
- Supports learning from unlabelled Digital Twin data before TGNN-based modelling.
- Our framework integrates SSL with TGNN and MARL for intelligent dynamic network slicing.

**Paper 5: Network-to-Network: Self-Supervised Network Representation Learning via Position Prediction (2025)**

*1. Problem Statement*

- Existing Graph Neural Networks (GNNs) rely heavily on labelled data, making training expensive and limiting scalability.
- Most network representation learning methods capture structural information but fail to preserve node positional information and rich node content simultaneously.
- The paper proposes a Self-Supervised Network Representation Learning framework (Net2Net) that learns meaningful node embeddings without requiring large labelled datasets.

***2. Methodology / Algorithm***

- Introduces Net2Net, a self-supervised encoder-decoder framework based on Node Position Prediction (PosPredict).
- Constructs ego networks for each node and learns embeddings by combining node content and graph topology.
- Uses a hierarchical GNN encoder and cross-modal decoder to predict each node's position (PosID), followed by semi-supervised fine-tuning for downstream tasks.

***3. Dataset / Simulation Tool***

- Evaluated on eight real-world graph datasets, including Cora, Citeseer, DBLP, PubMed, GitHub, Facebook, LastFM Asia, and Twitch.
- Compared against multiple graph embedding and GNN baselines such as DeepWalk, Node2Vec, GraphSAGE, GAT, DGI, and MVGRL.
- Performance was evaluated using node classification with Micro-F1 score.

***4. Results***

- Net2Net consistently achieved higher node classification accuracy than existing supervised and unsupervised methods.
- The proposed Position Prediction task produced more discriminative node embeddings by preserving structural, positional, and content information.
- Demonstrated strong generalization with minimal labelled data across diverse graph datasets.

***5. Research Gap***

- The framework learns static graph representations and does not capture temporal network evolution.
- Does not address Digital Twin modelling, dynamic network slicing, or reinforcement learning-based decision making.
- Primarily designed for node representation learning rather than autonomous network optimization.

***6. How it Relates to Our Project***

- Provides the Self-Supervised Learning foundation for learning graph representations from unlabelled network data.
- The learned embeddings can serve as inputs to our Temporal Graph Neural Network (TGNN) for modelling dynamic network states.

- Our project extends this work by integrating Digital Twins, TGNN, and MARL for intelligent dynamic network slicing in AI-native 6G networks.

**Paper 6: Encrypted Network Traffic Classification in SDN using Self-Supervised Learning (2022)**

***1. Problem Statement***

- Encrypted network traffic is difficult to classify using conventional supervised learning due to the need for large labelled datasets.
- Preparing labelled network traffic data is costly and prone to errors.
- The paper investigates Self-Supervised Learning (SSL) for accurate traffic classification in Software Defined Networks (SDNs).

***2. Methodology / Algorithm***

- Builds an SDN testbed using Mininet, Open vSwitch, and the RYU Controller to generate and collect network traffic.
- Uses a two-stage SSL framework with pre-training on unlabelled traffic followed by fine-tuning on a small labelled dataset.
- Extracts protocol-independent traffic features from flow statistics for real-time encrypted traffic classification.

***3. Dataset / Simulation Tool***

- Generated a custom SDN dataset containing five encrypted traffic classes: Counter Strike (Active/Idle), Quake III, DNS, and VoIP.
- Traffic was created using Mininet and the D-ITG traffic generator, while features were collected using the RYU SDN Controller.
- Both offline evaluation and real-time traffic classification experiments were conducted.

***4. Results***

- SSL achieved 97.64% classification accuracy, outperforming the supervised model by approximately 2%.
- Real-time traffic classification achieved up to 99% accuracy with an average inference time of approximately 0.002 seconds.
- Demonstrated that SSL effectively reduces label dependency while maintaining high classification performance.

***5. Research Gap***

- Focuses only on encrypted traffic classification in SDN environments.
- Does not perform network representation learning, Digital Twin modelling, or dynamic network slicing.
- The SSL framework is task-specific and not designed for graph-based network optimization.

***6. How it Relates to Our Project***

- Demonstrates that Self-Supervised Learning can effectively learn from large amounts of unlabelled network data.
- Provides evidence that SSL reduces dependence on manually labelled datasets, supporting the motivation for our SSL module.
- Our framework extends SSL beyond traffic classification by combining it with Digital Twins, TGNN, and MARLfor autonomous network slicing in AI-native 6G networks.

KRISH S

**Paper 1: Toward Enabling Network Slice Mobility to Support 6G System (2022)**

***Problem Statement***

- Future 6G networks will support diverse applications with different latency, bandwidth, and resource requirements.
- When users move, the VNFs providing their services may need to be migrated, scaled, created, or relocated to maintain the required QoS.
- Relocating VNFs closer to users can reduce end-to-end latency, but frequent relocation can cause service interruption.
- Therefore, the main problem is finding a balance between low end-to-end delay and minimum service relocation while maintaining the slice SLA

.

***2. Methodology***

The authors propose three Network Service Management solutions:

- DELAY-NSM: Minimizes end-to-end delay by placing VNFs closer to users through VNF instantiation, migration and resource reconfiguration.
- R-NSM: Minimizes VNF/service relocation to reduce service interruption while satisfying delay, bandwidth and jitter requirements.

- FT-NSM: Uses the Kalai-Smorodinsky bargaining game to obtain a Pareto-fair trade-off between minimizing delay and minimizing service relocation.

The framework can dynamically perform VNF scaling up/down, VNF creation/deletion, VNF migration, and UE reassignment

***3. Dataset***

- Uses a discrete-time network simulation, not a public dataset.
- Simulates UEs, eNB/gNBs, Edge Clouds, VNFs, and network slices.

***UE mobility is modeled using random walk with pymobility.***

***Tests different numbers of UEs and Edge Clouds.***

***Maximum end-to-end delay is set to 30 ms.***

***Each simulation is repeated 80 times for reliable results.***

***4. Results***

***DELAY-NSM: Lowest latency of 16–17.5 ms.***

***R-NSM: Lowest relocation time of 5–7.5 ms.***

- FT-NSM: Good balance — around 18 ms latency and 8–10 ms relocation time.
- FT-NSM provides the best overall trade-off, but has higher computational cost.

***5. Research Gap***

***Relies mainly on optimization and game theory.***

***Does not use SSL for learning network representations.***

***Does not use TGNN for future traffic prediction.***

- Does not use MARL for intelligent cooperative resource allocation.
- Does not use a Digital Twin for network simulation and learning.

***6. How it relates to our Project***

***Directly supports our dynamic 6G network slicing concept.***

- Shows the importance of dynamically scaling, migrating and relocating VNFs.
- Demonstrates the need to balance latency, QoS and resource management.
- Our project extends this using SSL + TGNN + Digital Twin + MARL for predictive and intelligent network slicing.

**Paper 2: Slicing for AI: An Online Learning Framework for Network Slicing Supporting AI Services (2025)**

***1.Problem Statement***

- 6G will support many AI-based services with different resource and QoS requirements.
- Static/offline slicing struggles with dynamic and time-varying network conditions.
- The goal is to dynamically allocate resources while improving AI model accuracy, latency, and cost

***2.Methodology***

- Proposes an Online Learning for Slicing (OLS) framework.
- Uses the EXP3 online learning algorithm to adapt resource allocation over time.
- Introduces OLS-SA and OLS-RSA to reduce the decision space and improve learning speed.
- Jointly allocates computing + communication resources and tunes AI model parameters.

***3.Dataset***

- Uses a mobile-health Deep Learning dataset with 245,921 samples.

***Dataset contains 13 physical-activity classes.***

- Tests different training-data sizes and numbers of epochs.

***Results are averaged over 10 experiments***

***4.Results***

- Proposed methods converge toward optimal resource allocation.
- OLS-SA and OLS-RSA reduce the decision space and computational complexity.
- Performs better under dynamic network conditions and varying resource availability

***5.Research Gap***

***Focuses mainly on slicing resources for AI services.***

***Does not use TGNN for future network-traffic prediction.***

***Does not use MARL for cooperative slicing decisions.***

- Does not integrate a Digital Twin or SSL-based network representation learning.

***6.How It Relates to Our Project***

***Strongly supports our dynamic 6G network slicing component.***

- Shows how learning can adapt resource allocation to changing network conditions.

- Our project extends this with SSL + TGNN + Digital Twin + MARL for predictive and autonomous network slicing.

**Paper 3: Self-Supervised Spatiotemporal Graph Neural Networks With Self-Distillation for Traffic Prediction (2023)**

***1.Problem Statement***

- Existing spatiotemporal GNNs struggle to capture wider spatial-temporal dependencies.
- Deeper GNNs can suffer from overfitting and poor generalization.
- The paper aims to improve traffic prediction accuracy and robustness.

***2. Methodology***

- Proposes SSGNN, combining Self-Supervised Learning + Spatiotemporal GNN + Self-Distillation.
- SSL reconstructs masked traffic signals to learn better features.
- GCN captures spatial relationships, while temporal convolutions capture time dependencies.
- Self-distillation improves model generalization and stability.

***3.Dataset***

***Uses 6 real-world traffic datasets.***

***METR-LA, PEMS-BAY – traffic speed.***

***PEMS03, PEMS04, PEMS07, PEMS08 – traffic flow.***

***Uses 1 hour of historical data to predict the next 1 hour.***

***4.Results***

- SSGNN achieved better or competitive performance against strong GNN baselines.
- Maximum improvements were 3.0% MAE, 5.2% RMSE and 3.8% MAPE.
- Shows that SSL and self-distillation improve traffic forecasting.

***5.Research Gap***

- Predicts road traffic, rather than 6G network traffic/slice demand.
- Prediction is the final objective; it does not use predictions for dynamic resource allocation.
- No closed-loop mechanism to convert predicted traffic into network slicing decisions.
- Therefore, it does not address proactive 6G slice orchestration.

***6.How It Relates to Our Project***

***Very relevant to our SSL + TGNN prediction stage.***

- Its idea of learning spatial-temporal patterns from traffic data can conceptually be adapted to 6G network nodes and traffic.
- In our project, these learned patterns help predict future network/slice demand.
- We then go further: TGNN prediction → MARL decision-making → dynamic network slicing inside the Digital Twin.

**Paper 4: Deep Reinforcement Learning for End-to-End Network Slicing: Challenges and Solutions (2023)**

***1. Problem Statement***

- End-to-end network slicing involves many configuration parameters across RAN, transport, core, edge, and cloud.
- Traditional model-based methods struggle with such complex and dynamic resource allocation.
- The paper investigates DRL for intelligent end-to-end slice orchestration.

***2. Methodology***

- Models resource orchestration as a Markov Decision Process (MDP).
- DRL observes network state and traffic load and decides resource allocation for each slice.
- Explores Safety DRL, Distributed/Multi-Agent DRL, and Imitation Learning.
- Proposes a three-layer architecture: Network → Orchestration → Intelligence (DRL).

***3. Dataset***

- Uses a real network slicing testbed, rather than a public dataset.
- Uses OpenAirInterface/FlexRAN, OpenDayLight, OpenFlow, Docker and USRP B210.
- DRL agents are implemented using PyTorch with 3-layer neural networks.

***4. Results***

- Safety DRL reduces SLA violations from up to 15% to about 1%.
- Distributed DRL improves scalability using multiple agents.
- Imitation Learning gives DRL a better starting policy and helps accelerate online learning.
- The paper shows that DRL is promising for automating end-to-end resource orchestration, while practical deployment still faces scalability and convergence challenges.

### *5. Research Gap*

- DRL mainly reacts to observed network states; our project adds TGNN-based future traffic prediction before slicing decisions.
- The paper identifies difficulty in handling huge heterogeneous network data; our SSL stage conceptually addresses this by learning useful representations from unlabeled network data.
- DRL learning through real-network interaction can be slow and potentially unsafe; our Digital Twin provides a virtual environment for training/testing decisions.

### *6. How It Relates to Our Project*

- Very closely related to our MARL + Dynamic Network Slicing stage.
- Its distributed DRL concept uses multiple agents that coordinate resource allocation, directly supporting our choice of MARL.
- Our project adds a predictive pipeline before these agents: SSL → TGNN → predicted network demand → MARL → dynamic slicing.
- Thus, this paper provides a strong foundation for the decision-making/resource-allocation part of our proposed 6G architecture.

### *Additional Papers*

**Paper 3: Toward Self-Optimizing 6G Networks Through Network Digital Twin Intelligence: A Real Network Traffic Evaluation (2026)**

### *1. Problem Statement*

- Existing Digital Twins mainly support monitoring and simulation rather than autonomous optimization.
- Manual network management cannot efficiently handle dynamic 6G traffic.
- The paper proposes an AI-native Digital Twin for prediction, anomaly detection, load balancing, and energy optimization.

### *2. Methodology / Algorithm*

- Uses KPIs such as traffic, latency, load, energy efficiency, and reliability.
- Predicts traffic using a rolling average over previous observations.
- Employs Isolation Forest for anomaly detection and MLP for latency prediction.
- Performs adaptive load balancing and energy optimization in a closed-loop framework.

### *3. Dataset / Simulation Tool*

***Uses the Telecom Italia Milan Grid Dataset.***

- Contains approximately 10,000 cells and 1.85 million traffic records.
- Evaluates Internet traffic and Voice/SMS traffic separately.
- Models trained using a 60/20/20 train-validation-test split.

### *4. Results*

- Achieved 96.7% Internet traffic prediction accuracy and 98.3% Voice/SMS prediction accuracy.
- Reduced network load by 25.5% and achieved 30.4% energy savings.
- Obtained high anomaly detection and prevention accuracy.
- Outperformed conventional rule-based optimization methods.

### *5. Research Gap*

- Uses traditional ML models (Isolation Forest and MLP) instead of graph learning.
- Traffic prediction relies on a simple rolling average.
- Evaluation is limited to one dataset and a few KPIs.

### *6. How it Relates to Our Project*

- Demonstrates AI-driven Digital Twin intelligence for autonomous network management.
- Our framework replaces conventional ML with SSL and TGNN for richer network representations.
- MARL enables collaborative and adaptive network slicing beyond rule-based optimization.

### *Datasets*

| **Dataset / Tool** | **Purpose** |
|---|---|
| Telecom Italia Milan | Real cellular traffic forecasting |
| NeversNet5G | 5G wireless state + mobility + QoS |
| B5G Network Slicing Dataset | eMBB/URLLC/mIoT slicing + QoS |

**1. Telecom Italia Milan Mobile Traffic Dataset**

Telecom Italia dataset on Harvard Dataverse

Milan Grid dataset on Harvard Dataverse

Scientific Data paper describing the dataset

***What it contains***

This is one of the most important datasets for your project.

It contains anonymized mobile-network activity from Milan, aggregated into spatial grid cells and 10-minute intervals.

The telecom activity includes:

- SMS-in
- SMS-out
- Call-in
- Call-out
- Internet activity
- Grid ID
- Timestamp
- Country code

The original dataset covers November 1, 2013 to January 1, 2014, with the city represented spatially by a grid. Nature

The Milan grid is approximately 100 × 100 spatial cells, with cells around 235 m × 235 m. PLOS

**Why it is extremely useful for you**

***This is almost tailor-made for the TGNN part.***

***Critical limitation***

It does not contain actual 5G slice allocation, PRB allocation, latency constraints or MARL actions.

So don't pretend that Milan alone represents a 6G network.

***Instead, use it as your real-world traffic-demand layer.***

***2. NeversNet5G***

This is one of the most interesting datasets I found for your project because it is very recent.

NeversNet5G project page

NeversNet5G dataset on Hugging Face

Zenodo DOI for NeversNet5G

***What it contains***

NeversNet5G is a city-scale 5G NR vehicular network dataset generated using:

SUMO: +  Simu5G / OMNeT++

It contains approximately:

***390 vehicle trajectories***

***19 real-world base-station locations***

***4 network-load scenarios***

***242+ million records***

- SINR
- CQI
- throughput
- latency
- vehicle position
- speed
- road segment
- UE identity
- network measurements

The released data contains 242,304,261 rows across 716 per-UE CSV files. 6G-TWIN

**Why this is extremely interesting for your Digital Twin**

You essentially get:

Mobility

+

5G network state

+

Base stations

+

SINR

+

CQI

+

Throughput

+

Latency

That gives you a much richer state representation than Milan.

You can construct:

Graph:

gNB ——— gNB

|: |

|: |

UE ———— UE

and feed temporal network states into your TGNN.

***Big limitation***

It is generated through simulation, rather than being raw operational 5G measurements.

***3. B5G Network Slicing Dataset (Farreras et al., 2024)***

Link: https://zenodo.org/records/10610616 (paper: https://doi.org/10.1016/j.dib.2024.110738)

- Generated through a packet-level simulator, it captures network slicing considering the three main 3GPP slice types: eMBB, URLLC, and mIoT, across a wide range of scenarios with varying topologies, slice instances, and traffic flows. Each sample pairs a network configuration (topology, traffic characteristics, routing) with performance metrics (delay, jitter, loss) per flow, and includes deliberately over- and under-provisioned scenarios. Zenodonih
- Why it fits: it's already labeled by your exact three slice types (eMBB/URLLC/mMTC), gives you real topologies to build your TGNN's graph structure on, and the over/under-provisioning cases are perfect negative examples for training your MARL reward function to avoid QoS violations.

***What it contains***

This dataset was specifically generated for 5G/B5G network slicing.

It contains:

- network topology
- traffic characteristics
- routing configurations
- eMBB
- URLLC
- mIoT
- delay
- jitter
- packet loss
- flow-level performance

The dataset was generated using packet-level network simulation and includes scenarios with different levels of over/under-provisioning. ScienceDirect

**Why it fits your project**

This is almost exactly the environment you need for:

MARL state

↓

Agent action

↓

Resource allocation

↓

Network

↓

Delay / jitter / loss

↓

Reward

You can formulate:

***Agent state***

Traffic demand

Available bandwidth

Current utilization

Slice type

Latency

Packet loss

Previous allocation

***Action***

Allocate bandwidth

Allocate resources

Change slice allocation

***Reward***

Something like:

Reward =

QoS satisfaction

+ resource utilization

- latency penalty

- packet-loss penalty

- SLA violation penalty

***Major limitation***

The dataset models the transport network, not the complete RAN.

The paper explicitly states that the scenarios exclude the RAN infrastructure. ScienceDirect

So this should be your slicing/control benchmark, not your entire 6G dataset.

***Datasets***

***Dataset: Purpose***

***Telecom Italia: Model spatial-temporal traffic demand***

***OAI-KPM / TRACTOR: 5G/O-RAN network-state KPIs***

ns-3/OAI simulation: MARL actually perform slice allocation.

***1.Dataset: Telecom Italia – Telecommunications (SMS, Call, Internet) – Milan***

***Main dataset — Harvard Dataverse***

Telecom Italia Milan Dataset

***Milan spatial grid — Harvard Dataverse***

Milan Grid Dataset

***Scientific Data paper***

***Dataset Description Paper***

The dataset provides historical mobile-network activity aggregated over geographical grid cells at 10-minute intervals.

It includes:

- SMS-in activity
- SMS-out activity
- Call-in activity
- Call-out activity
- Internet activity
- Spatial grid/cell identifier
- Time interval
- Country code

The original dataset covers November 1, 2013 to January 1, 2014. Milan is represented using a regular spatial grid with cells of approximately 235 m × 235 m.

***Why we selected it***

This dataset provides a real-world representation of spatial and temporal variations in cellular traffic demand.

For our project, it can be used to:

- Identify traffic patterns across different geographical locations
- Model changing network demand over time
- Construct spatial relationships between network regions

***Provide input for TGNN-based traffic prediction***

- Generate future traffic-demand states for the Digital Twin

The spatial grid is particularly useful because neighbouring geographical cells can be represented as nodes/relationships in a graph.

***How it fits our Digital Twin***

***Telecom Italia***

↓

***Historical traffic demand***

↓

***Spatial-temporal representation***

↓

***Digital Twin traffic state***

↓

***TGNN***

↓

***Future traffic prediction***

***Important limitation***

The dataset does not provide actual 5G/6G network-control information, such as:

- PRB allocation
- RAN resource utilization
- Slice configuration
- Network latency
- MARL actions
- QoS/SLA decisions

Therefore, we use Telecom Italia primarily as the real-world traffic-demand layer, rather than claiming that it represents a complete 6G network.

***2.OAI-KPM / TRACTOR 5G-O-RAN Telemetry***

For the second layer, we use 5G/O-RAN network telemetry to complement the traffic-demand information.: OAI-KPM:

- Official GitHub repository + dataset: OAI-KPM Dataset
- Direct dataset file: OAI-KPM Excel Dataset

***Dataset documentation/README: OAI-KPM Documentation***

OAI-KPM provides RAN metrics such as latency, PRB usage, PDCP volume, RLC delay and UE throughput from an OAI experimental 5G setup.

TRACTOR:

- Official GitHub repository: TRACTOR O-RAN Repository

***Dataset/logs: available under the repository's logs/ and raw/ directories.***

- TRACTOR project/research page: TRACTOR Project Page

There are two closely related options here:

***OAI-KPM***

The OAI-KPM dataset was collected using a bare-metal OpenAirInterface experimental setup consisting of OAI 5G Core, OAI gNB, FlexRIC and a KPM xApp. It provides approximately 5 weeks of RAN time-series measurements.

It contains network-state variables such as:

- UE identifier/registration
- Network latency
- Downlink PRB usage
- Uplink PRB usage
- PDCP data volume
- RLC delay
- UE throughput
- Uplink/downlink measurements

It contains network-state variables such as:

- UE identifier/registration
- Network latency
- Downlink PRB usage
- Uplink PRB usage
- PDCP data volume
- RLC delay
- UE throughput
- Uplink/downlink measurements

Latency

PRB utilization

Throughput

RLC delay

↓

Network state

***TRACTOR***

TRACTOR provides another useful O-RAN telemetry source. It collected 447 minutes of real 5G traffic from a Google Pixel 6 Pro, covering different applications, locations and mobility conditions. Those traffic traces were then replayed in a full-stack srsRAN-based O-RAN environment on the Colosseum RF emulator to generate O-RAN KPI data.

Its released KPI dataset contains 17 selected O-RAN KPIs, including:

- Downlink MCS
- Downlink buffer size
- Downlink throughput
- CQI
- Uplink MCS
- Uplink buffer size
- Uplink throughput
- Uplink SINR
- Requested PRBs

- Granted PRBs
- UE power headroom

The KPIs are collected at 250 ms intervals through the O-RAN E2 interface.

**Why we need this layer**

Telecom Italia tells us:

***"How much traffic demand exists and where/when it occurs?"***

OAI-KPM / TRACTOR tells us:

"What is happening inside the 5G network while handling that traffic?"

Therefore, this layer allows the Digital Twin to represent network conditions such as:

***Traffic demand***

+

***PRB utilization***

+

***Throughput***

+

***Latency***

+

***Channel quality***

+

***Buffer status***

↓

***Digital Twin network state***

***3.ns-3 / OpenAirInterface Controlled Network Environment***

**ns-3 / OAI Simulation Environment**

For the third component, this isn't really a dataset like Telecom Italia or OAI-KPM. It's your controllable simulation/emulation environment.

- Official ns-3 website: ns-3 Network Simulator
- Official ns-3 App Store: ns-3 App Store

- OpenAirInterface: OpenAirInterface 5G/6G Software Alliance

ns-3 is an open-source discrete-event network simulator intended for research and education.

The first two sources provide observational data, but our MARL agent requires something different: an environment that can respond to the agent's actions.

This is where ns-3/OAI comes in.

***What it provides***

We can construct a controlled 5G/B5G network environment containing:

- Base stations/gNBs
- UEs
- Network traffic
- Radio resources
- Bandwidth
- Network links
- QoS requirements
- Different traffic/service classes
- Network congestion conditions

ns-3 also has 5G/mmWave capabilities and supports cellular-network simulation features such as channel models, MAC/PHY components, RLC and cellular mobility.

**Why simulation is necessary**

MARL works through an interaction loop:

Current Network State

↓

MARL Agent

↓

Action

↓

Resource/Slice Allocation

↓

Network Environment

↓

New Network State

↓

Reward

↓

Agent learns

A static historical dataset cannot naturally perform this state → action → new state → reward interaction.

Therefore, simulation gives us the controllable environment required to train and evaluate the MARL-based dynamic slicing policy.

**How the Three Work Together:**
This is the most important part of your methodology.

REAL / EXPERIMENTAL DATA



↓: ↓

Telecom Italia: OAI-KPM / TRACTOR

Traffic Demand: Network Telemetry



↓

DIGITAL TWIN

↓

Self-Supervised Learning

↓

TGNN

↓

Future Traffic/Network State

↓

MARL

↓

ns-3 / OAI Environment

↓

Dynamic Slice Allocation

↓

QoS + Resource State

↓

DIGITAL TWIN

↺

Telecom Italia provides the real-world spatial-temporal traffic demand, OAI-KPM/TRACTOR provides experimental 5G/O-RAN network-state telemetry, and ns-3/OAI provides the controllable environment in which the MARL agent can learn and evaluate dynamic network-slicing decisions.

# Appendix B: Columns of the Digital Twin Dataset

**Identity and calendar (20)**
scenario, interval, day_index, date, interval_of_day, hour, minute, weekday, month, day_of_year, iso_week, season, is_weekend, is_holiday, is_business_hours, is_night, source, source_date, cell_id, zone

**Control (3)**
policy, idle_share, reconfiguration_churn

**Events and state (8)**
event_type_code, event_type, event_id, anomaly, cell_asleep, capacity_factor, interference_factor, anomaly_score

**Cell radio and capacity (11)**
capacity_mbps, spare_capacity_mbps, total_demand_mbps, total_served_mbps, prbs_used, load, sinr_db, cqi, modulation, spectral_efficiency_bps_hz, all_slices_sla_ok

**Neighbourhood (6)**
neighbour_load, neighbour_mean_demand_mbps, neighbour_all_sla_ok, rehomed_in_mbps, rehomed_out_mbps, handover_users_est

**Energy (3)**
power_w, energy_wh, energy_efficiency_mbit_per_j

**Per slice, 21 fields for each of eMBB, URLLC and mMTC (63)**
share, prbs, demand_mbps, demand_next_10min_mbps, demand_next_30min_mbps, demand_next_60min_mbps, demand_1h_mean_mbps, demand_delta_mbps, users_est, allocated_mbps, utilisation, served_mbps, throughput_per_user_mbps, packets, queue_kbit, latency_p50_ms, latency_p95_ms, jitter_ms, loss, sla_margin_ms, sla_ok

**Reward (2)**
reward, team_reward

**Telemetry as observed (6)**
obs_total_demand_mbps, obs_embb_latency_p95_ms, obs_load, obs_sinr_db, obs_cqi, telemetry_missing